Seer Transport Fleet Logistics LiveStack Guide

Introduction

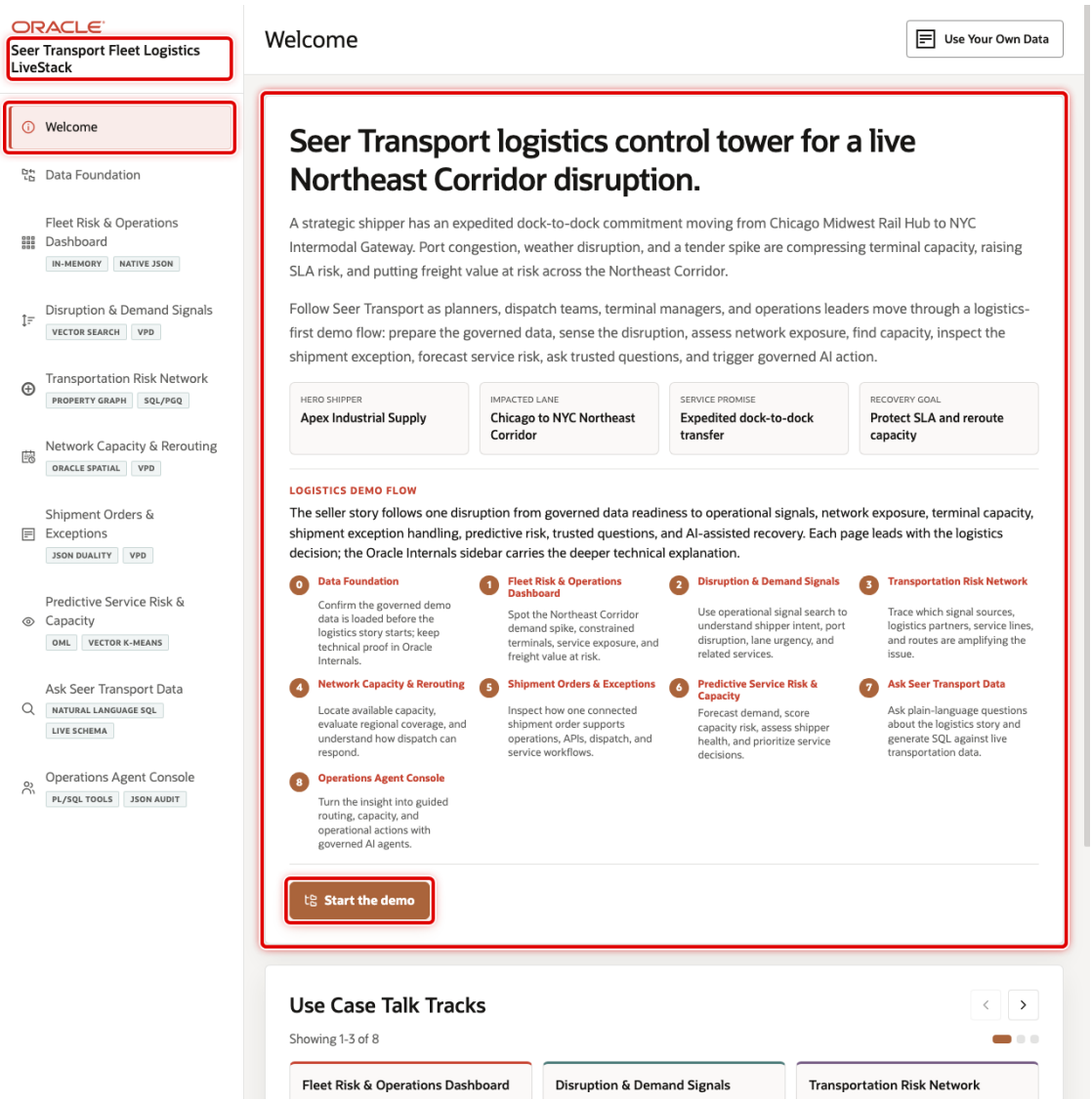
Transportation and logistics teams need to make faster decisions while shipment orders, terminal capacity, disruption signals, routes, service commitments, equipment constraints, and customer expectations are spread across many systems. The Seer Transport Fleet Logistics LiveStack shows how a transportation operator can bring those signals together, understand what needs attention, and move from insight to action with more confidence.

This runbook supports the Seer Transport Fleet Logistics LiveStack Demo. The demo shows how Oracle AI Database 26ai can help transportation teams bring those workloads together on one connected data foundation. Instead of splitting relational transactions, JSON documents, graph relationships, spatial analysis, vector search, machine learning, natural-language SQL, and AI agent workflows across separate systems, the LiveStack shows how those capabilities can work against the same governed Oracle data model.

In the demo, Seer Transport uses Oracle AI Database to connect transportation services, shipment orders, shippers, terminals, disruption and demand signals, risk networks, spatial routing, predictive analytics, conversational data access, and agent-assisted operations. The demo follows a logistics-first Northeast Corridor disruption thread: demand builds, shipper and port signals explain why, terminal capacity and shipment order views show how the business responds, and analytics, natural-language SQL, and AI agents help teams act from the same governed data foundation.

Estimated Demo Time: 90 minutes

Each scene is designed to take between 5 and 10 minutes.



Seer Transport LiveStack welcome page

Objectives

In this LiveStack demo, you will see how a transportation operator can use connected data and AI-assisted workflows to identify disruption risk, protect shipment commitments, improve terminal and route decisions, and make analytics easier to use.

Prerequisites

Before you begin, confirm that you can open the running Seer Transport Fleet Logistics LiveStack in a modern browser. No database or coding knowledge is required to follow the business workflow.

Demo Flow

- Scene 1: Welcome and Demo Orientation.

- Scene 2: Data Foundation.
- Scene 3: Fleet Risk & Operations Dashboard.
- Scene 4: Disruption & Demand Signals.
- Scene 5: Transportation Risk Network.
- Scene 6: Network Capacity & Rerouting.
- Scene 7: Shipment Orders & Exceptions.
- Scene 8: Predictive Service Risk & Capacity.
- Scene 9: Ask Seer Transport Data.
- Scene 10: Operations Agent Console.

Learn More

- Oracle AI Database 26ai documentation
- Oracle AI Agent Memory
- Oracle AI Vector Search
- Oracle Spatial and Graph documentation: Oracle Spatial and Oracle Property Graph
- Oracle Machine Learning for SQL documentation
- Oracle REST Data Services documentation
- Oracle LiveLabs catalog

Credits & Build Notes

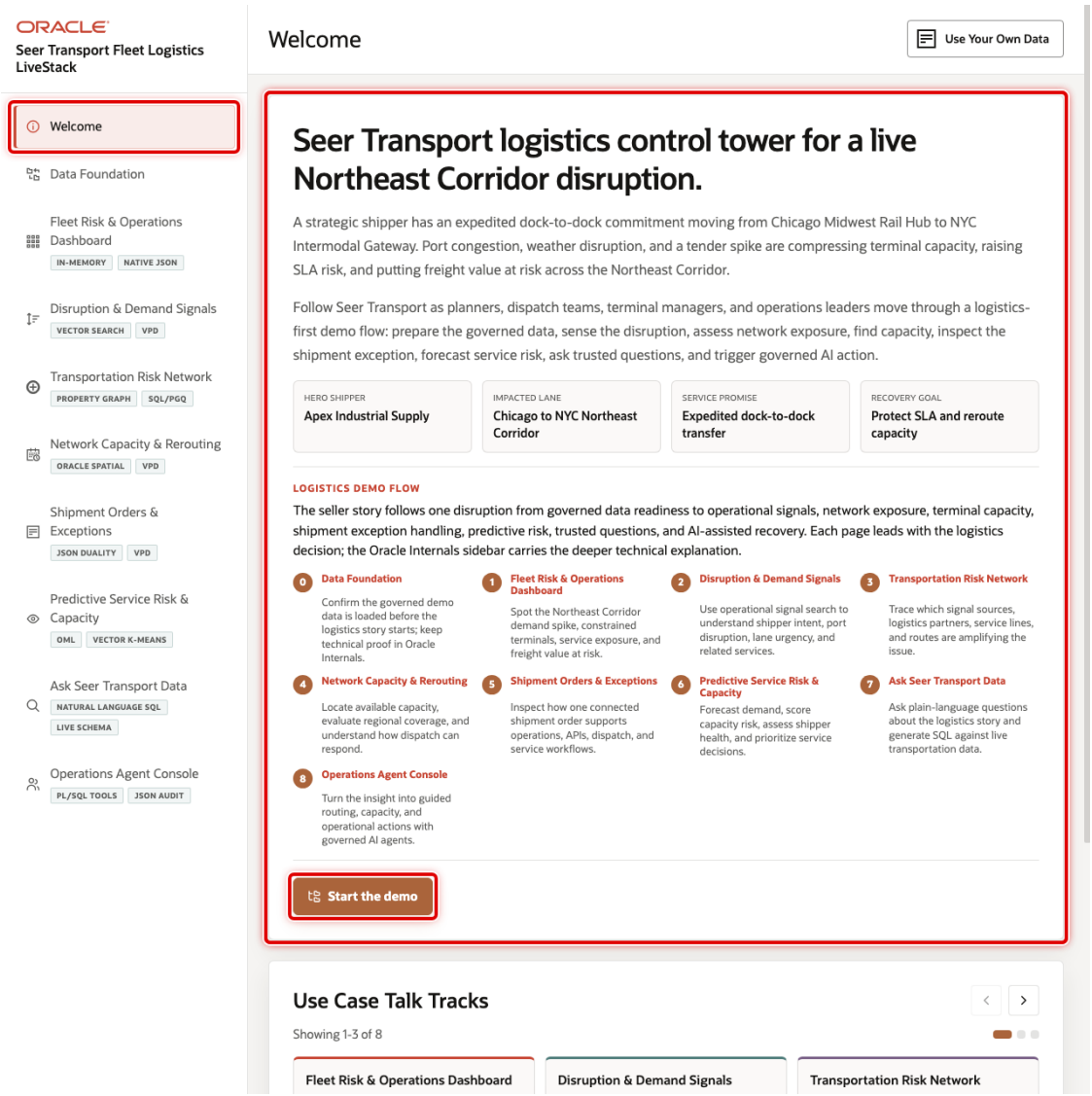
- **Author** - Oracle LiveLabs Team
- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-05-29

Scene 1 Welcome and Demo Orientation

Introduction

This opening scene orients users to the Seer Transport Fleet Logistics LiveStack Demo. The welcome page introduces the Northeast Corridor disruption story, identifies the hero shipper and impacted lane, and connects the left navigation to the operational journey that follows.

Estimated Time: 5 minutes



Welcome page with the logistics story and demo journey highlighted

Objectives

In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

Task 1: Review the logistics story and demo journey

Review the welcome page first so the audience understands the full transportation journey. The page frames the demo around an expedited dock-to-dock commitment from Chicago Midwest Rail Hub to NYC Intermodal Gateway, with port congestion, weather disruption, and a tender spike compressing capacity.

1. Review the opening story at the top of the page.

2. Review the four scenario cards: **Hero shipper**, **Impacted lane**, **Service promise**, and **Recovery goal**.
3. Review the numbered demo journey. It connects the same disruption thread to data readiness, signals, network exposure, capacity, shipment exceptions, predictive risk, trusted questions, and AI-assisted action.
4. Review the visible use case tiles in **Use Case Talk Tracks**.
5. Click the right carousel arrow to move forward through the remaining use cases.
6. Use the left carousel arrow if you want to return to earlier tiles.

Task 2: Continue the demo

1. Click **Start the demo**.
2. Confirm the demo moves to **Data Foundation**.

Credits & Build Notes

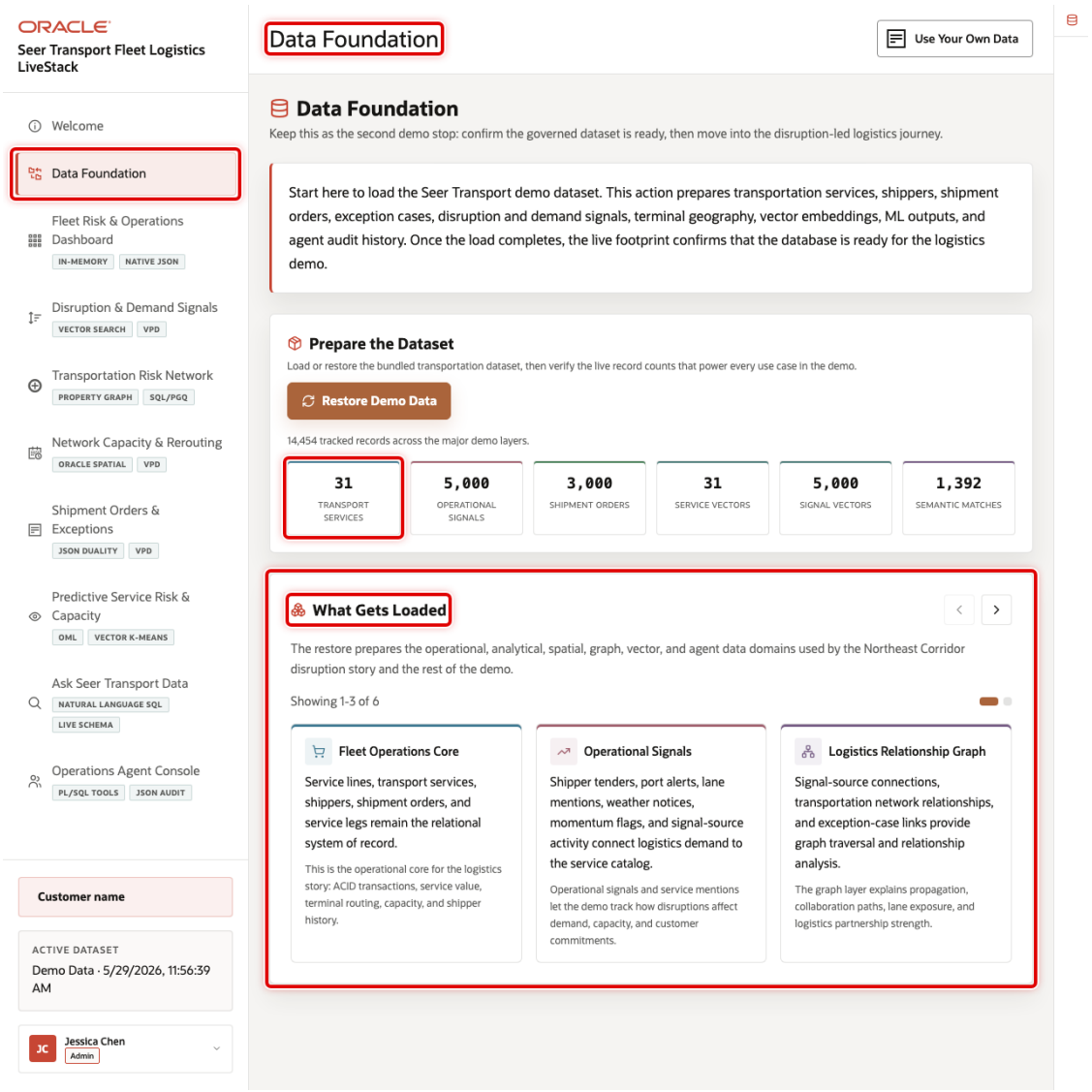
- **Author** - Oracle LiveLabs Team
- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-05-29

Scene 2 Data Foundation

Introduction

Data Foundation confirms that the governed demo dataset is loaded before the logistics story begins. It shows the transportation services, shippers, shipment orders, operational signals, terminal geography, vector embeddings, machine-learning outputs, and agent audit history that support every later scene.

Estimated Time: 5 minutes



Data Foundation page with the dataset action, live counts, and data foundation carousel highlighted

Objectives

In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

Task 1: Prepare the dataset

Use this page to show that the demo is backed by live seeded data rather than static screenshots. In the current demo dataset, the live footprint includes **31** transportation services, **5,000** operational signals, **3,000** shipment orders, **12** logistics terminals, **360** demand forecasts, and **5,000** signal embeddings.

1. Click **Data Foundation** in the sidebar.

2. Review the dataset preparation panel.
3. Click **Load / Restore Demo Data** if the dataset needs to be refreshed.
4. Review the live footprint cards after the page finishes loading.

Task 2: Review what gets loaded

1. Review the carousel of capability groups.
2. Use the right carousel arrow to move through fleet operations, operational signals, graph, spatial, JSON duality, predictive analytics, and agent capabilities.
3. Review the **How the Data Connects** section to show how each later scene uses the same governed Oracle data foundation.
4. Open Oracle Internals only after the business flow is clear. Use it to connect the visible logistics data to the database capabilities behind the page.

Credits & Build Notes

- **Author** - Oracle LiveLabs Team
- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-05-29

Scene 3 Fleet Risk & Operations Dashboard

Introduction

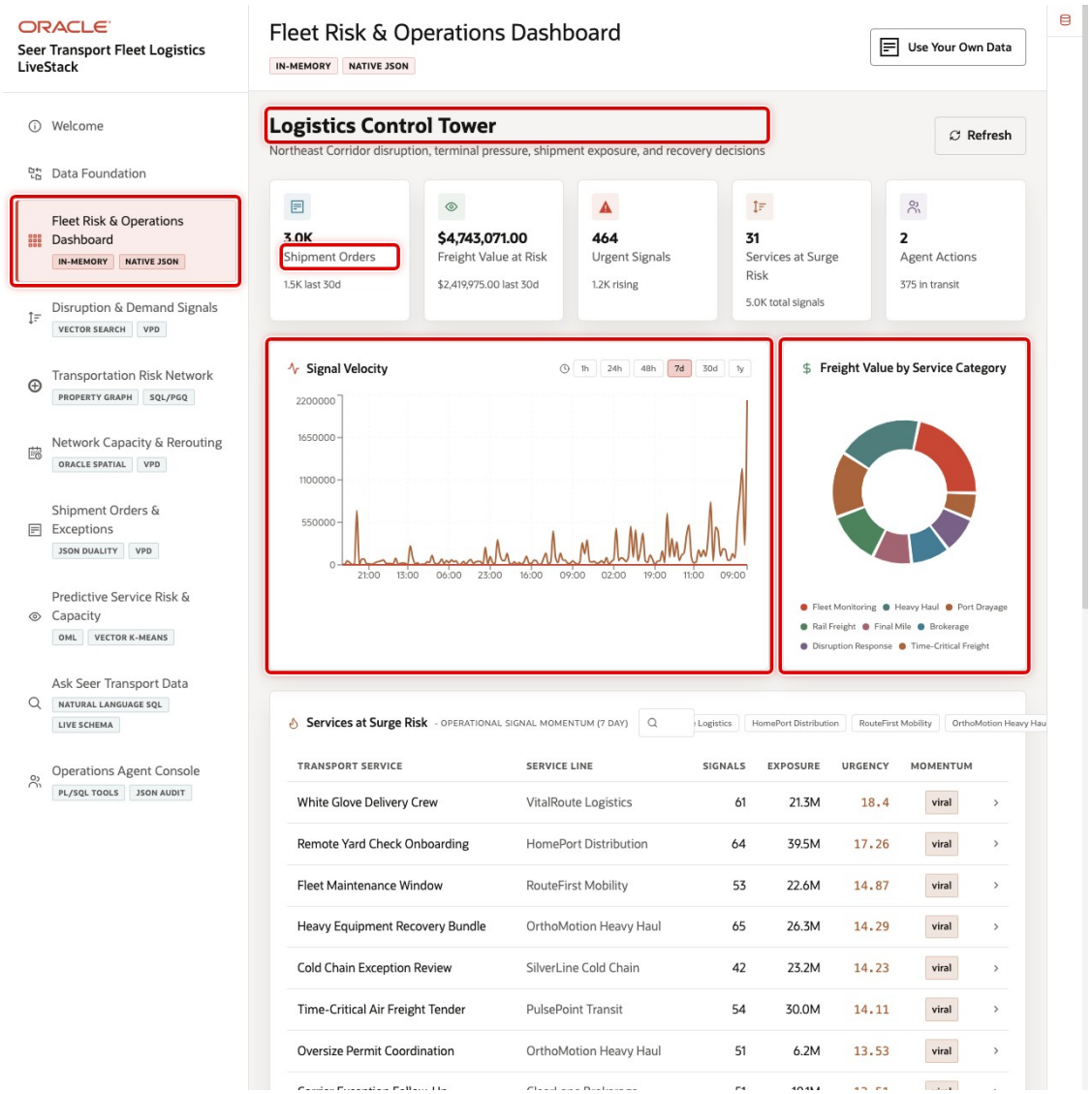
Fleet Risk & Operations Dashboard gives transportation leaders, dispatch planners, and service operations managers a daily view of the logistics network. It tracks shipment orders, freight value exposure, urgent operational signals, services at surge risk, in-transit shipments, signal velocity, category value, and service-level detail in one place.

This team must see shifts in demand, capacity, and service risk as they emerge, not after they become separate escalations. The dashboard helps leaders spot patterns early, act faster, and keep dispatch, terminal, and customer teams aligned around the same recovery decision.

Dashboards like this are difficult to implement when transportation data is split across order systems, fleet tools, terminal spreadsheets, social or partner signal feeds, and analytics pipelines. Oracle AI Database helps address that challenge by keeping operational, analytical, JSON, in-memory, and AI-ready data close to the same governed data foundation.

The **White Glove Delivery Crew** row gives users a clear opening example: demand is visible at the dashboard level and then traceable down to freight value, service category, signal exposure, and JSON application shape. The dashboard also defines **Signal Velocity** as the rate and intensity of new disruption, demand, and service-risk signals over the selected time window, so the term is clear without extra explanation.

Estimated Time: 10 minutes



Fleet Risk & Operations Dashboard with KPI cards, charts, and services at surge risk highlighted

Objectives

In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

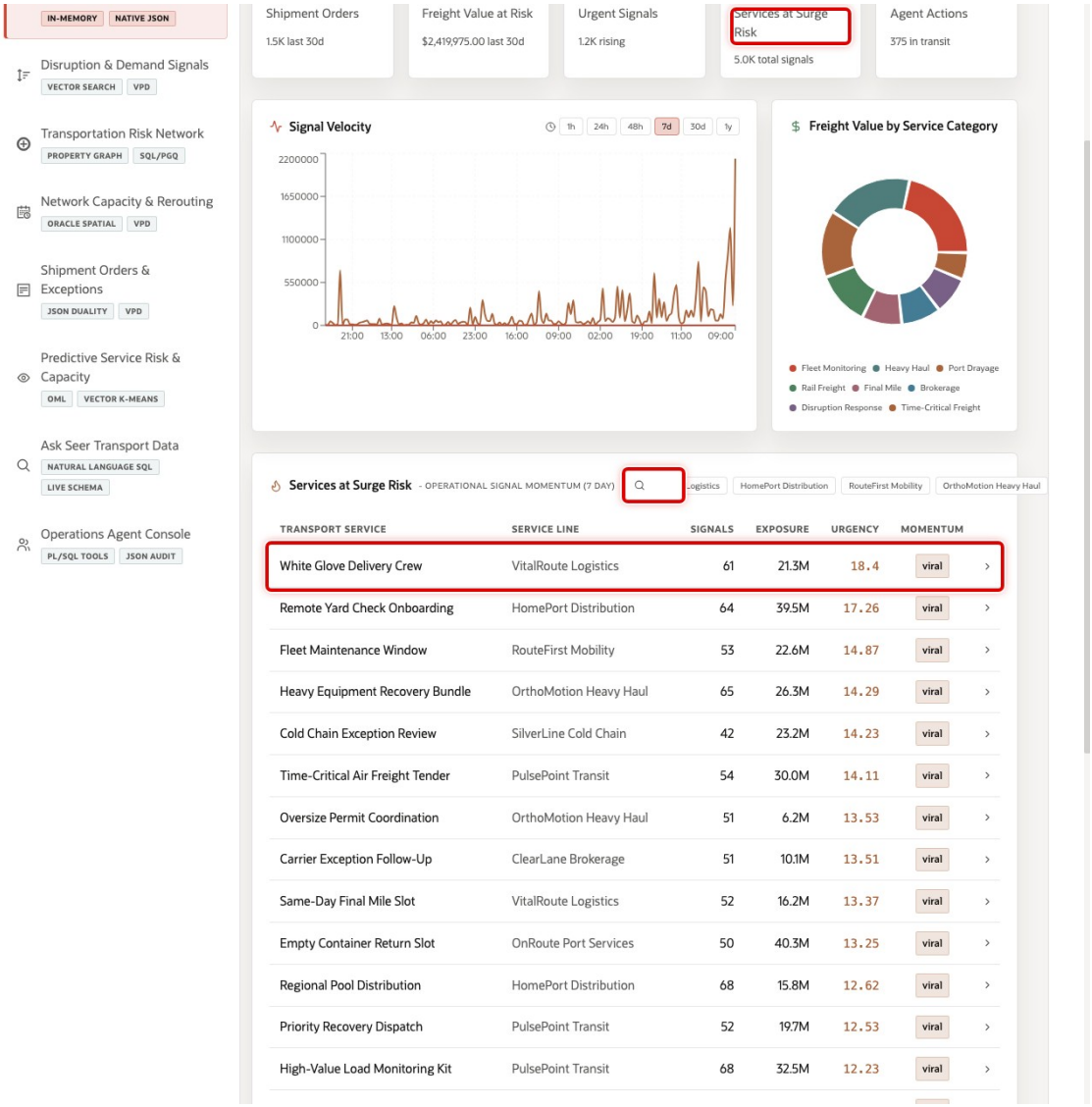
Task 1: Review the control tower dashboard

Use the dashboard as a triage view. In the current demo dataset, the opening KPI row shows **3,000** shipment orders, about **$4.7M** in freight value at risk, **464** urgent signals, **31** services at surge risk, and **375** shipments in transit.

1. Click **Fleet Risk & Operations Dashboard** in the sidebar.
2. Review the KPI cards across the top of the page.

3. Review **Signal Velocity** to see how quickly new disruption, demand, and service-risk signals are building.
4. Review **Freight Value by Service Category** to see where value exposure is concentrated across related transportation services. Explain that this is a concentration view, not a statistical correlation chart.
5. Review Oracle Internals after the business flow is clear. Use it to connect the visible logistics outcome to the database capabilities behind the page.

Task 2: Review services at surge risk



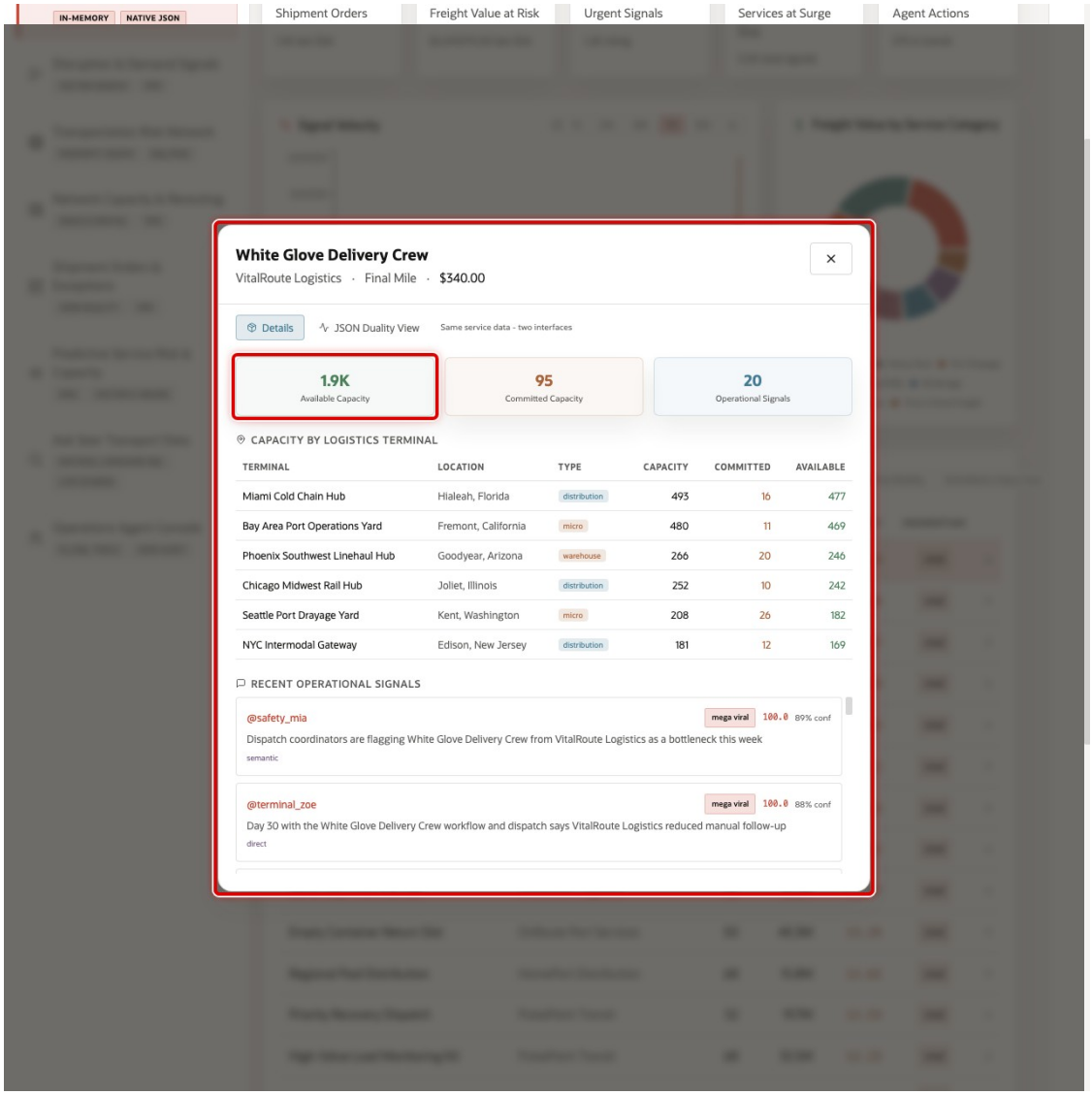
Services at surge risk table with White Glove Delivery Crew highlighted

The table helps the user move from dashboard-level signals to service-level evidence. In the current demo dataset, **White Glove Delivery Crew** appears as a leading VitalRoute Logistics service with **61** recent mentions, more than **21M** views, and **viral** peak momentum.

1. Scroll to **Services at Surge Risk**.

2. Review the service rows. The table ranks services by recent operational signal momentum and shows service name, service line, mentions, views, average urgency, and signal label.
3. Use the search field or service-line chips if you want to narrow the table.
4. Click the **White Glove Delivery Crew** row.

Task 3: Inspect the service detail modal

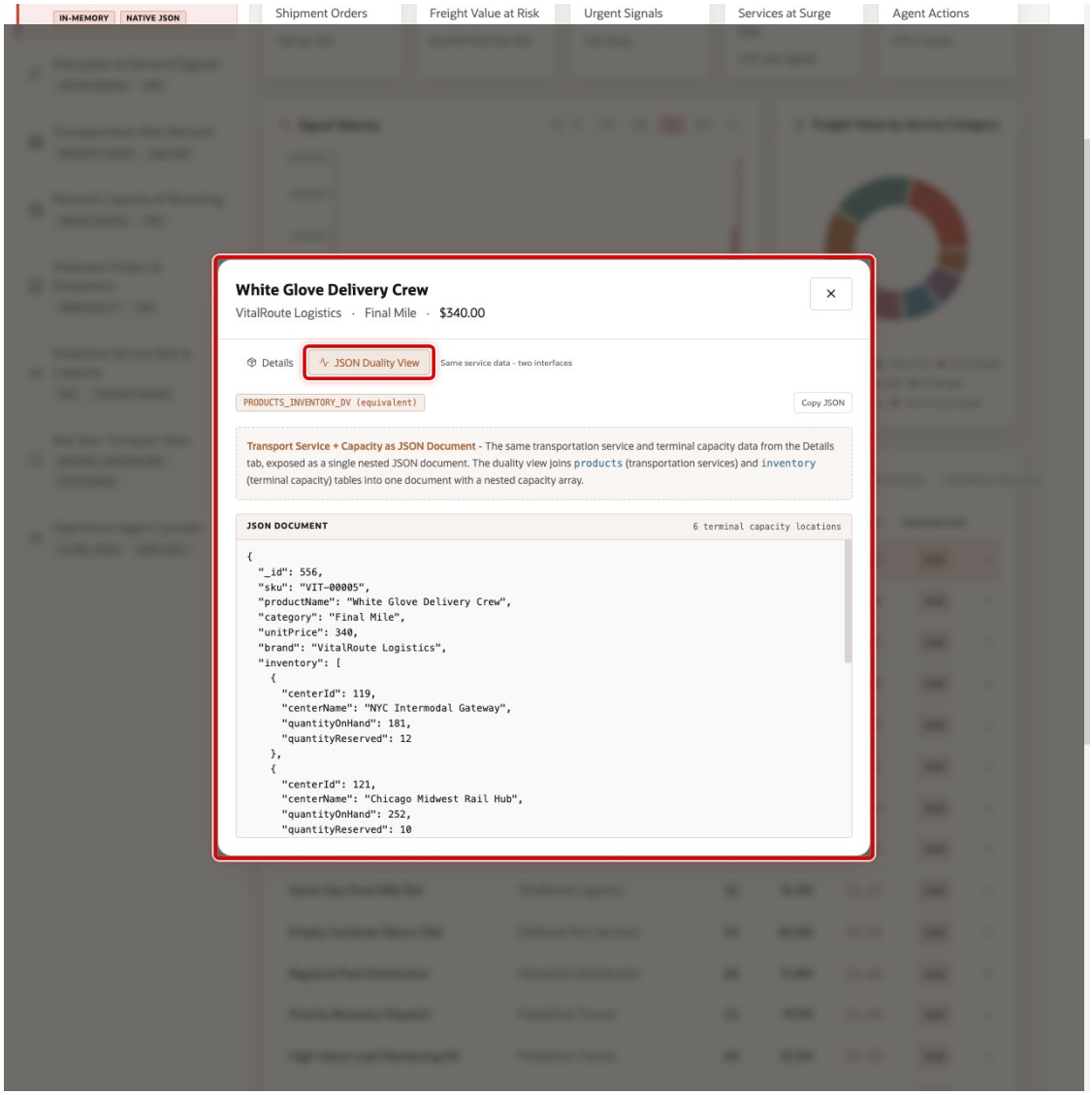


Transport service detail modal

Open the service detail modal to connect demand momentum with operational readiness. The user can see whether the selected service has enough terminal capacity and whether recent signals support further action.

After you click **White Glove Delivery Crew**, the detail modal opens. The default **Details** view shows the selected service, VitalRoute Logistics service line, Final Mile category, contract rate, terminal capacity locations, and operational signal evidence.

Task 4: Review the JSON Duality View



Transport service JSON Duality view

Review the JSON Duality View to show that the same trusted service and capacity data can support different users. Business users see service details in the interface, while applications can use the same information as a structured document.

1. In the service modal, click **JSON Duality View**.
2. Review the JSON document that represents the selected transportation service and terminal capacity data.

You can move to the next scene.

Credits & Build Notes

- **Author** - Oracle LiveLabs Team

- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-06-09

Scene 4 Disruption & Demand Signals

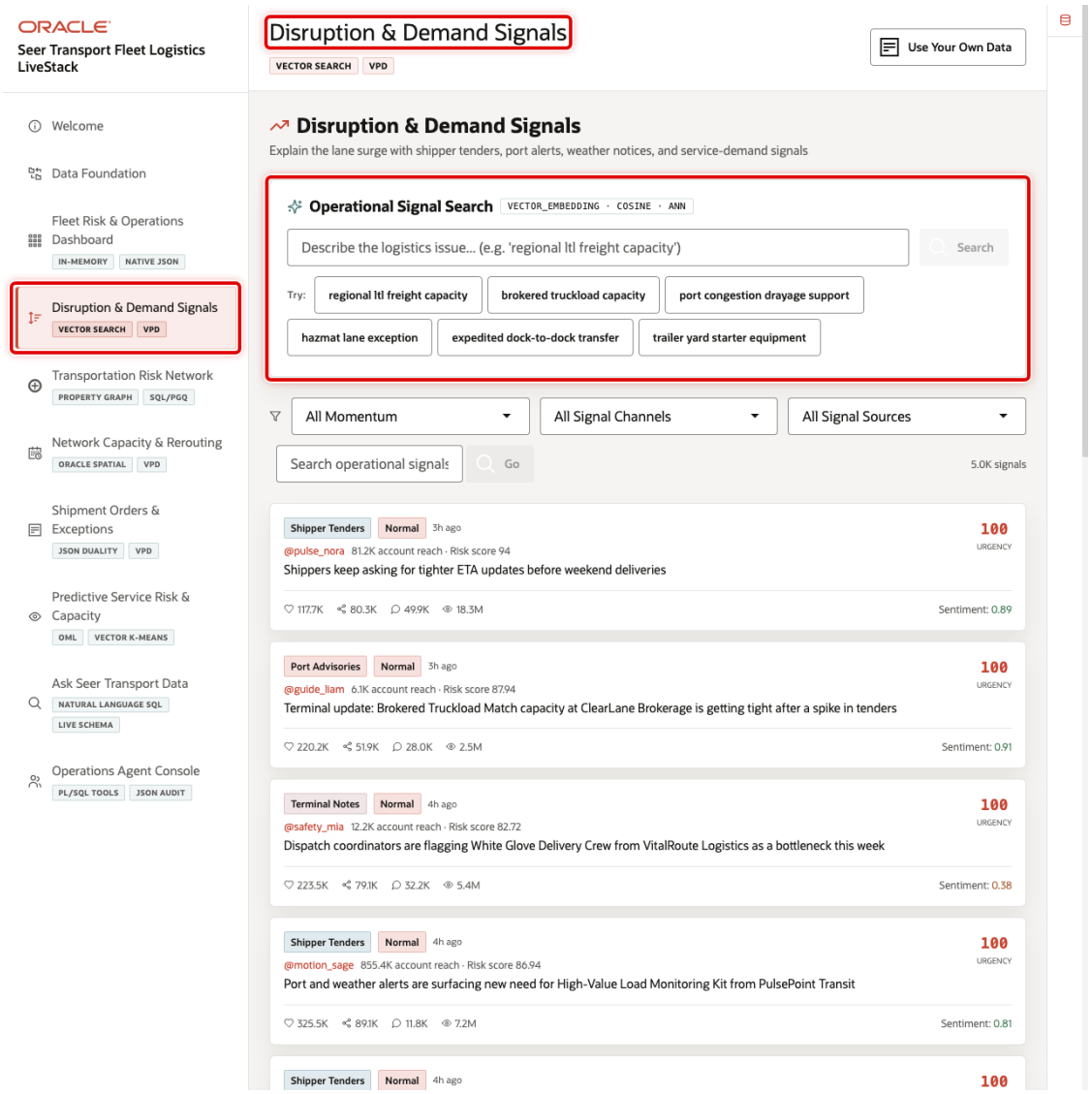
Introduction

Disruption & Demand Signals helps transportation planners explain why a surge is happening. The page combines semantic service discovery with operational signal intelligence from shipper tenders, port advisories, partner updates, weather notices, and lane-pressure indicators.

Transportation teams often receive signals faster than they can classify them. A terminal note, port advisory, or shipper tender may indicate real demand pressure, but it is difficult to connect that text to the affected services, regions, shippers, or terminals when signal data is outside the operational system.

Oracle AI Database helps by keeping vectors, signal text, service metadata, and row-level security close to the same governed data foundation. In this scene, operators can search in plain language, review ranked semantic matches, filter operational signals, and inspect the evidence that explains a service surge.

Estimated Time: 10 minutes



Disruption & Demand Signals page with service vector search and operational signal feed highlighted

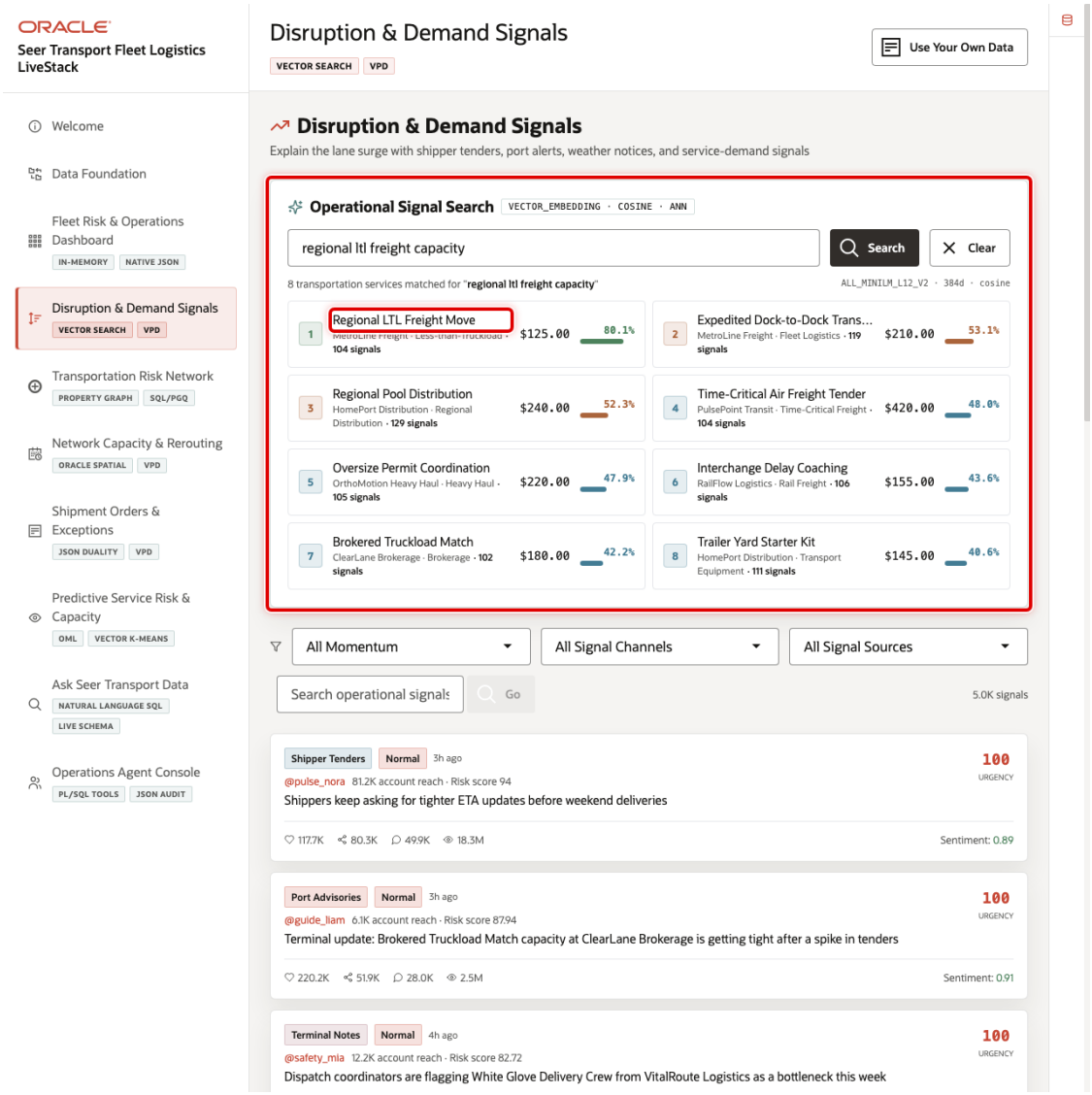
Objectives

In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

Task 1: Review the signal intelligence workspace

1. Click **Disruption & Demand Signals** in the sidebar.
2. Review **Operational Signal Search** and the available example query buttons.
3. Review the signal filters for momentum, channel, and signal source.
4. Review the operational signal feed.

Task 2: Run operational service discovery

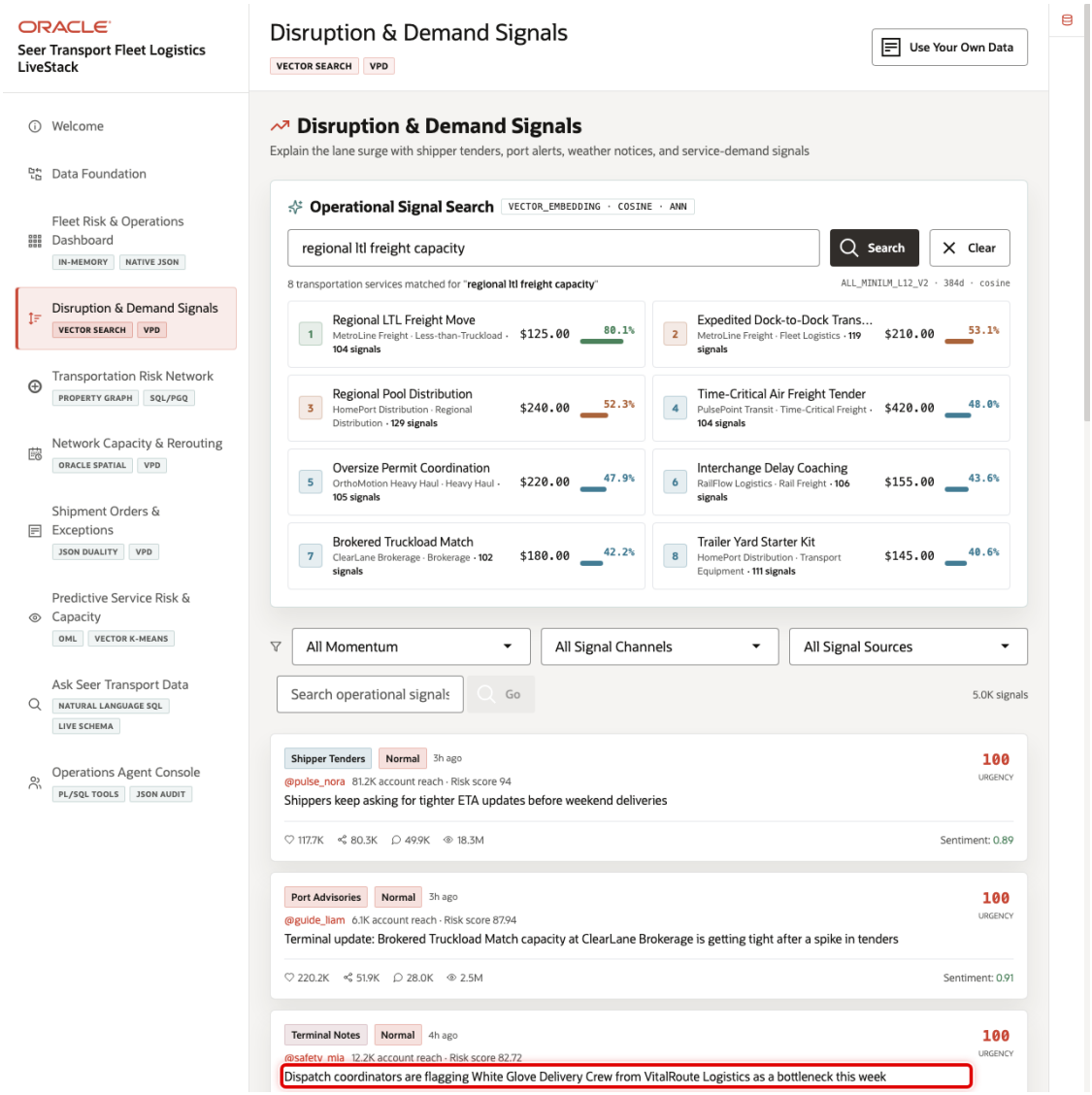


Operational service vector search results

Use semantic search to show that planners can describe a logistics need without knowing the exact service name.

1. In **Operational Signal Search**, enter `regional ltl freight capacity`.
2. Click **Search**.
3. Review the ranked services, similarity percentages, service lines, categories, and contract rates.
4. Point out that Oracle AI Vector Search can return relevant transportation services even when the search phrase is not an exact product name.

Task 3: Review operational signal evidence

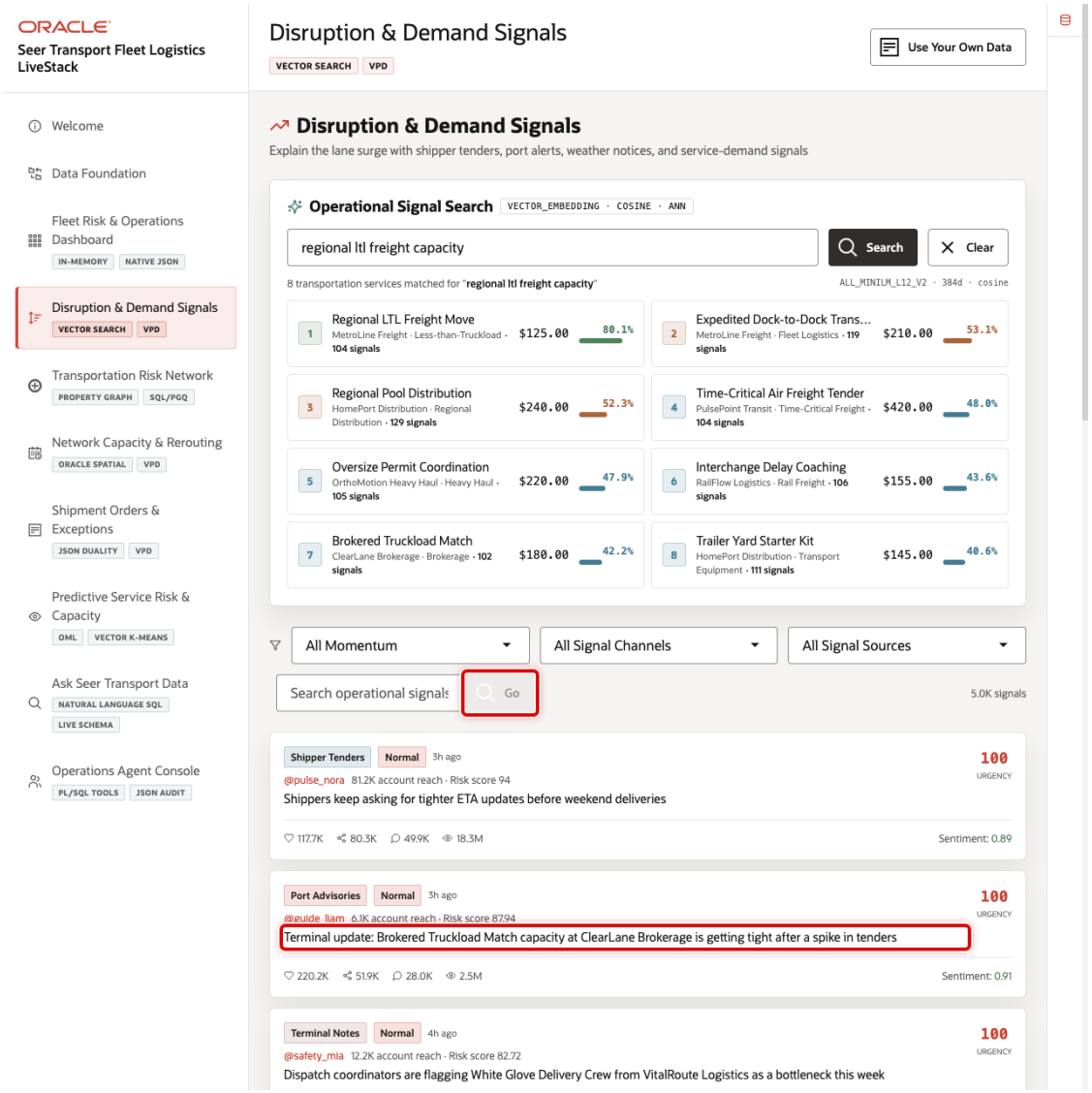


Operational signal feed with urgent transportation signals highlighted

Use the feed to connect demand movement to real signal evidence. In the current demo dataset, recent signals include tight ETA requests, terminal updates, and a **White Glove Delivery Crew** bottleneck alert.

1. Use the momentum filter to focus the feed on urgent or critical signals.
2. Use the signal channel filter to narrow the feed if needed.
3. Review signal text, urgency, reach, source, and affected service details.

Task 4: Run a signal evidence query



Operational signal query results

Run a text search to show how the signal feed can answer a focused operational question.

1. In **Search posts by embedding**, enter `terminal capacity delay`.
2. Click **Go**.
3. Review the returned signal rows and match percentages.
4. Use the results to explain which operational signals should be investigated before they turn into missed service commitments.

You can move to the next scene.

Credits & Build Notes

- **Author** - Oracle LiveLabs Team

- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-05-29

Scene 5 Transportation Risk Network

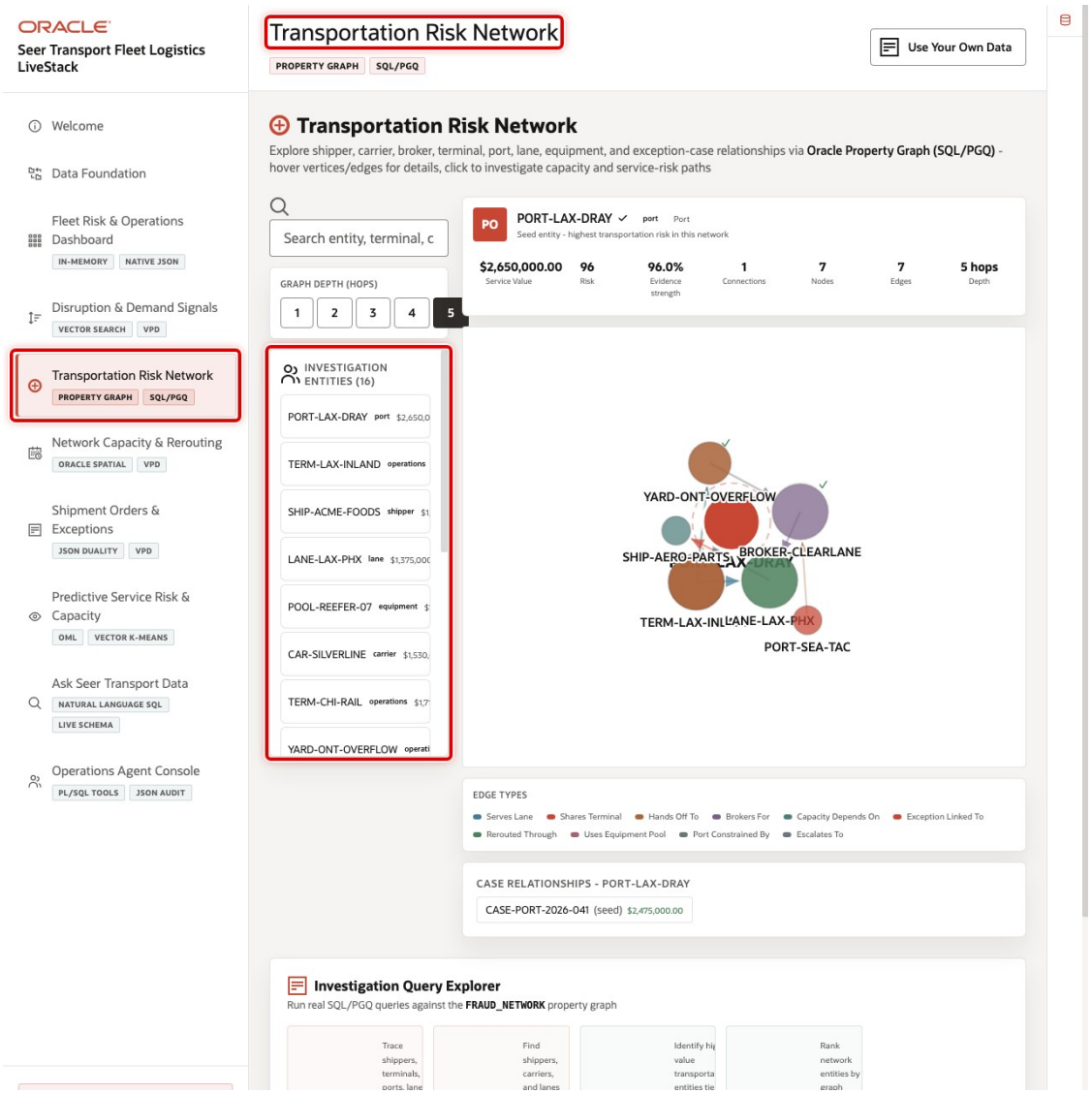
Introduction

Transportation Risk Network helps operations teams trace how disruption risk propagates across shippers, terminals, ports, carriers, routes, yards, equipment pools, and exception cases. It turns isolated signal evidence into a relationship view that can explain why one disruption affects multiple services.

Flat tables can show a delayed shipment or a constrained terminal, but they do not easily explain how that problem connects to carrier partners, shared terminals, lanes, equipment pools, or open exception cases. Transportation teams need a way to follow those relationships without rebuilding a network model outside the operational database.

Oracle AI Database helps by supporting graph-style relationship analysis against the same governed transportation data. In this scene, users can inspect a default risk network, select a transportation entity, and run graph query examples that reveal influence, shared dependency, and exception propagation. The graph opens with a less congested default depth so the network is easier to understand before expanding it.

Estimated Time: 10 minutes



Transportation Risk Network page with graph workspace and entity list highlighted

Objectives

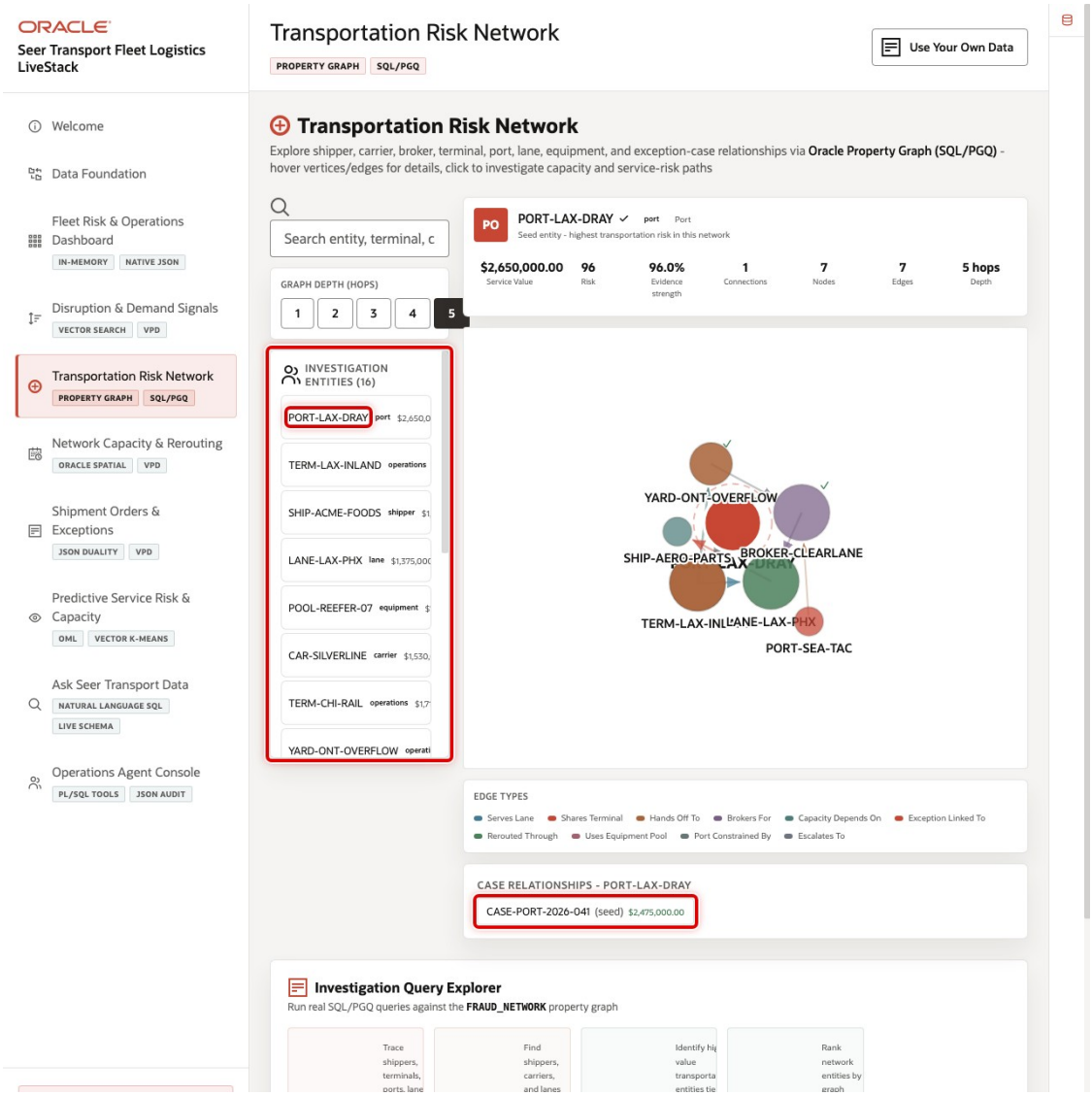
In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

Task 1: Review the risk network workspace

1. Click **Transportation Risk Network** in the sidebar.
2. Review the graph visualization, the larger **Edge Types** legend, and the list of investigation entities.
3. Review the selected entity details. In the current demo dataset, **PORT-LAX-DRAY** is a critical port entity with a **96** risk score, **96%** evidence strength, and **$2.65M** in service value.

4. Change the graph depth control only after the default network is clear; higher depth reveals more relationships and can make the network denser.

Task 2: Inspect the selected network data point

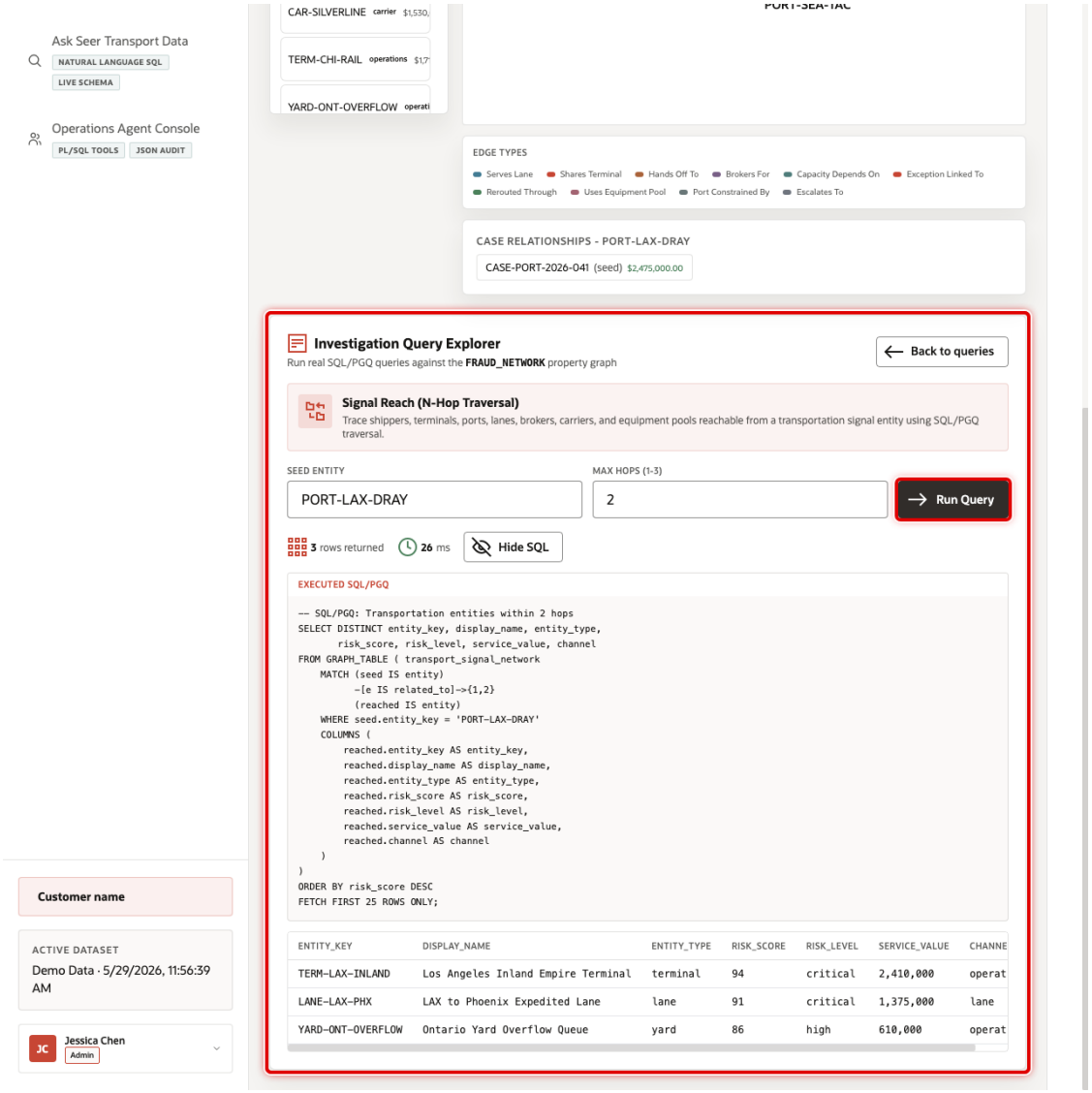


Selected transportation risk network data point

Use the selected entity details to make the graph concrete. The data point should show a named port, terminal, shipper, route, or carrier rather than a generic node.

1. Select **PORT-LAX-DRAY** or another visible critical entity.
2. Review its risk score, entity type, evidence strength, connection count, hop depth, and case relationships.
3. Explain how a constrained port or terminal can affect lanes, carriers, and shipment commitments downstream.

Task 3: Run a graph query example



Transportation risk network query results

Run a graph query example to show that the visible network is backed by queryable relationship data rather than a static diagram.

1. Open the graph query examples area.
2. Select a query such as influence reach, mutual connections, or community hub detection. The updated query cards use more width so the descriptions are easier to scan.
3. Click the run action for the selected query.
4. Review the wider result table and connect the returned rows to the graph view.

You can move to the next scene.

Credits & Build Notes

- **Author** - Oracle LiveLabs Team
- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-06-09

Scene 6 Network Capacity & Rerouting

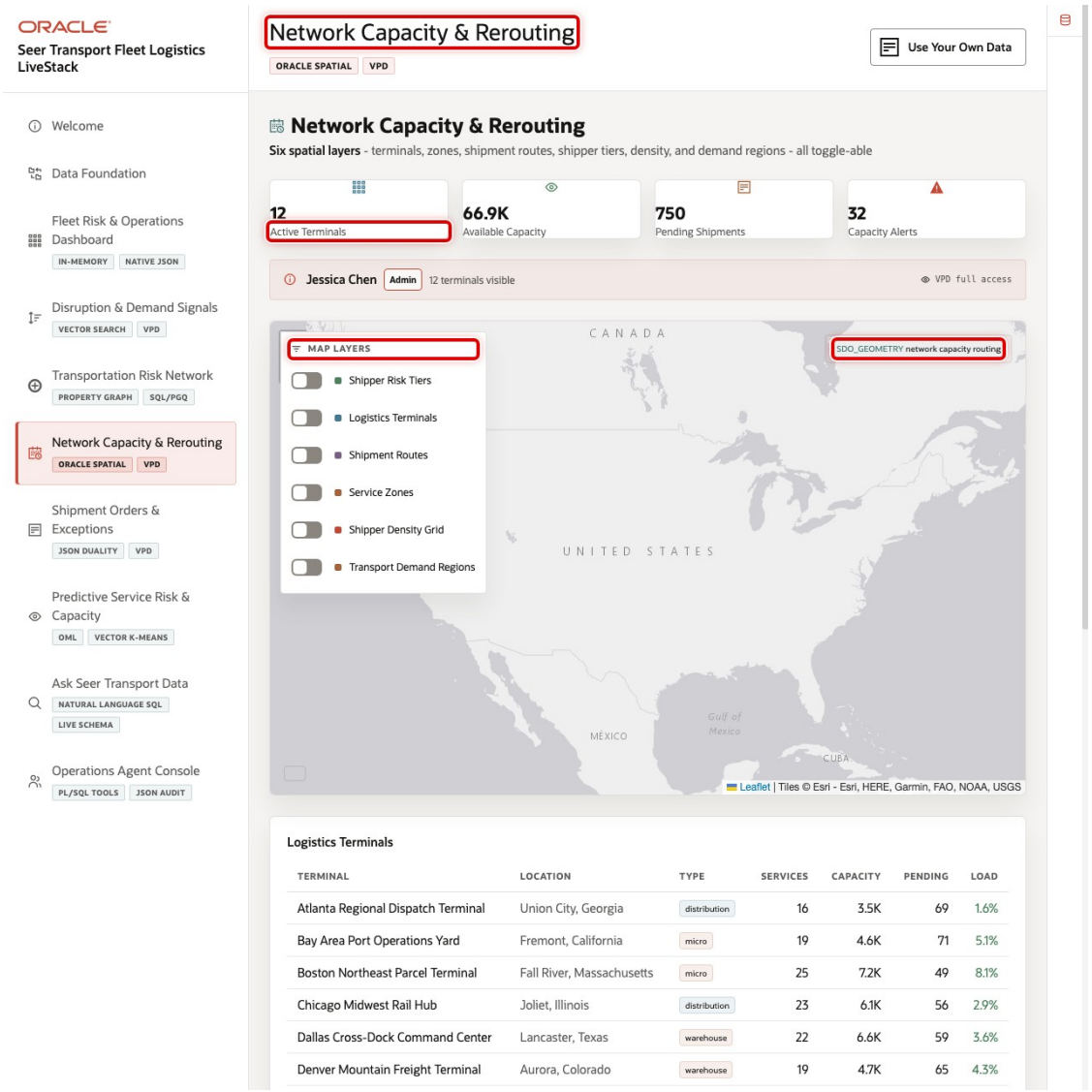
Introduction

Network Capacity & Rerouting helps dispatch planners, terminal managers, and network operations teams decide where the logistics network can absorb overflow. It combines terminal coverage, service zones, shipment routes, shipper density, demand regions, capacity alerts, and spatial context in one map-driven workspace.

Transportation capacity decisions depend on geography. A planner needs to know which terminal is close enough, whether a service zone covers the affected customer, which lanes are under pressure, and whether a terminal has enough available capacity. These questions are difficult when maps, capacity tables, and demand forecasts live in separate systems.

Oracle AI Database helps by keeping spatial geometry, terminal capacity, shipment routes, demand forecasts, and operational tables together. In this scene, users can toggle spatial layers, isolate a single shipment route, inspect terminal load, and investigate capacity alerts without leaving the application.

Estimated Time: 10 minutes



Network Capacity & Rerouting overview with KPIs and map highlighted

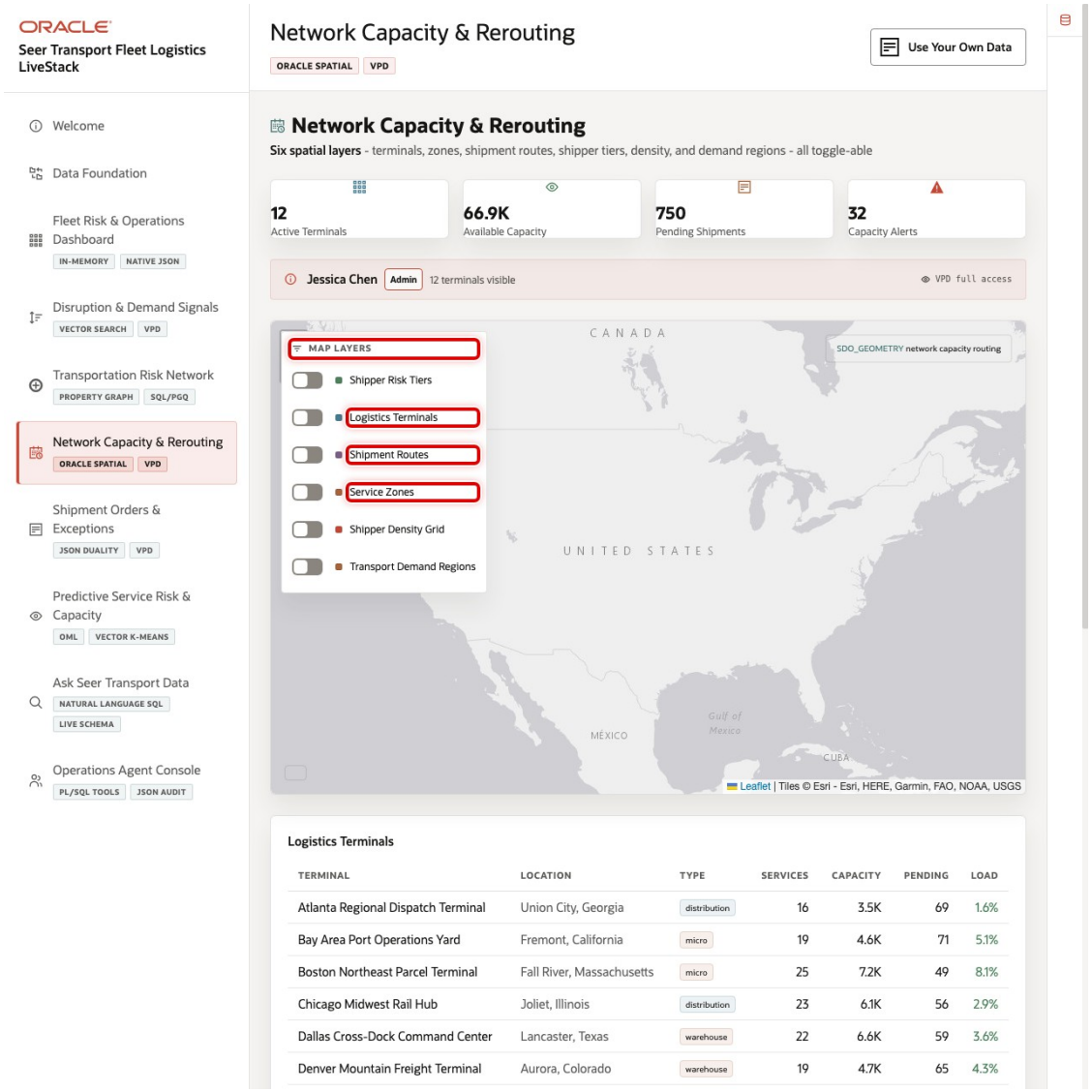
Objectives

In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

Task 1: Review the network capacity context

1. Click **Network Capacity & Rerouting** in the sidebar.
2. Review the KPI cards for active terminals, available capacity, pending shipments, and capacity alerts.
3. Review the map and confirm that Oracle Internals is collapsed unless you are explaining the technical implementation.

Task 2: Explore spatial demand and service coverage

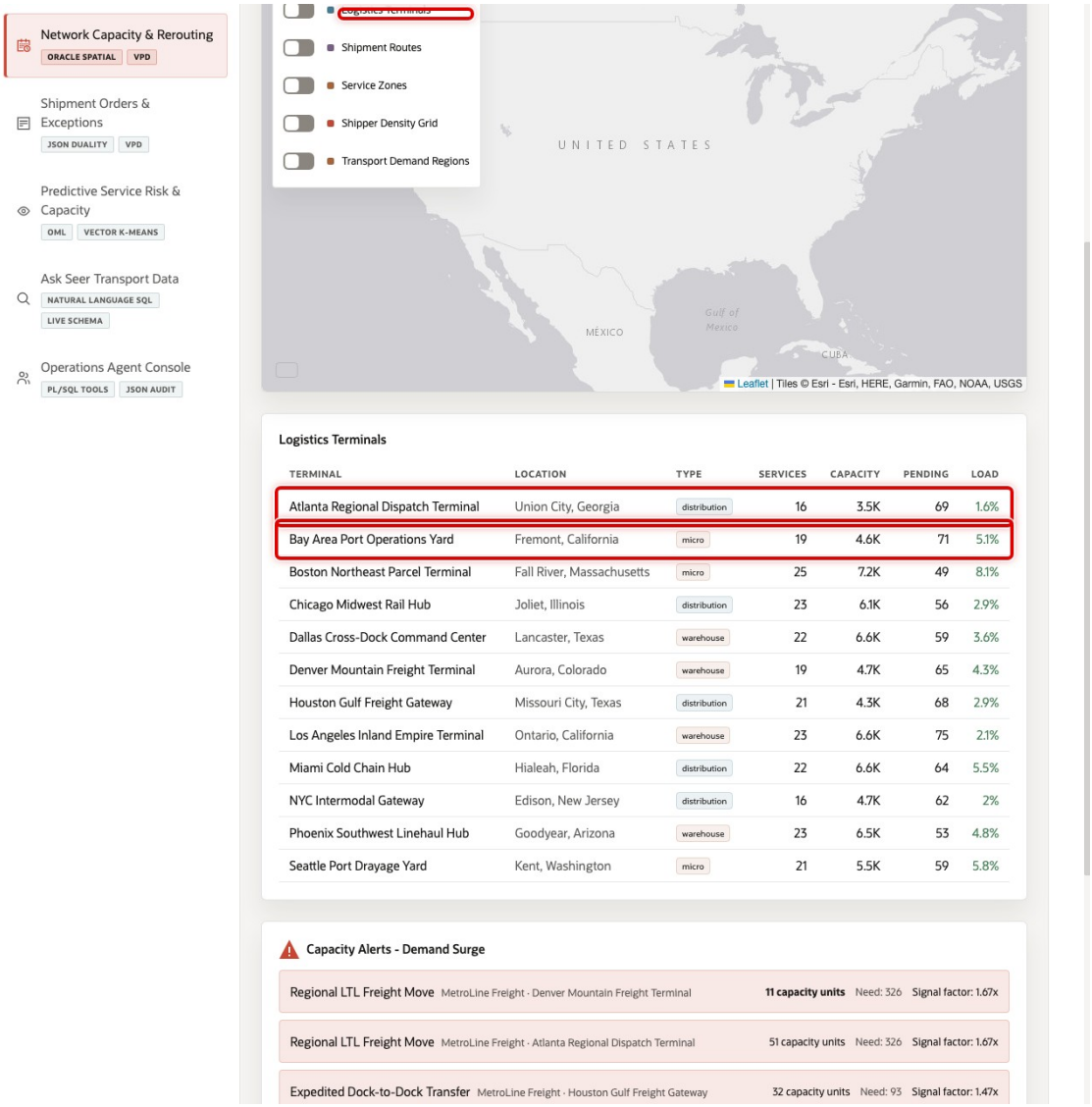


Spatial demand and service-zone layers highlighted

Use the map layers to show that terminal routing decisions can combine coverage, demand, and route context.

1. Turn on **Logistics Terminals**.
2. Turn on **Service Zones** and compare coverage areas.
3. Turn on **Shipment Routes**. Use the **Shipment Route** selector in the map layer panel to isolate one route when overlapping lines make the full route set hard to read.
4. Turn on **Shipper Density Grid** and **Transport Demand Regions**.
5. Hover over or click a map item to inspect the operational details, including carrier, order, origin terminal, destination, distance, and status for a focused route.

Task 3: Inspect terminal capacity rows

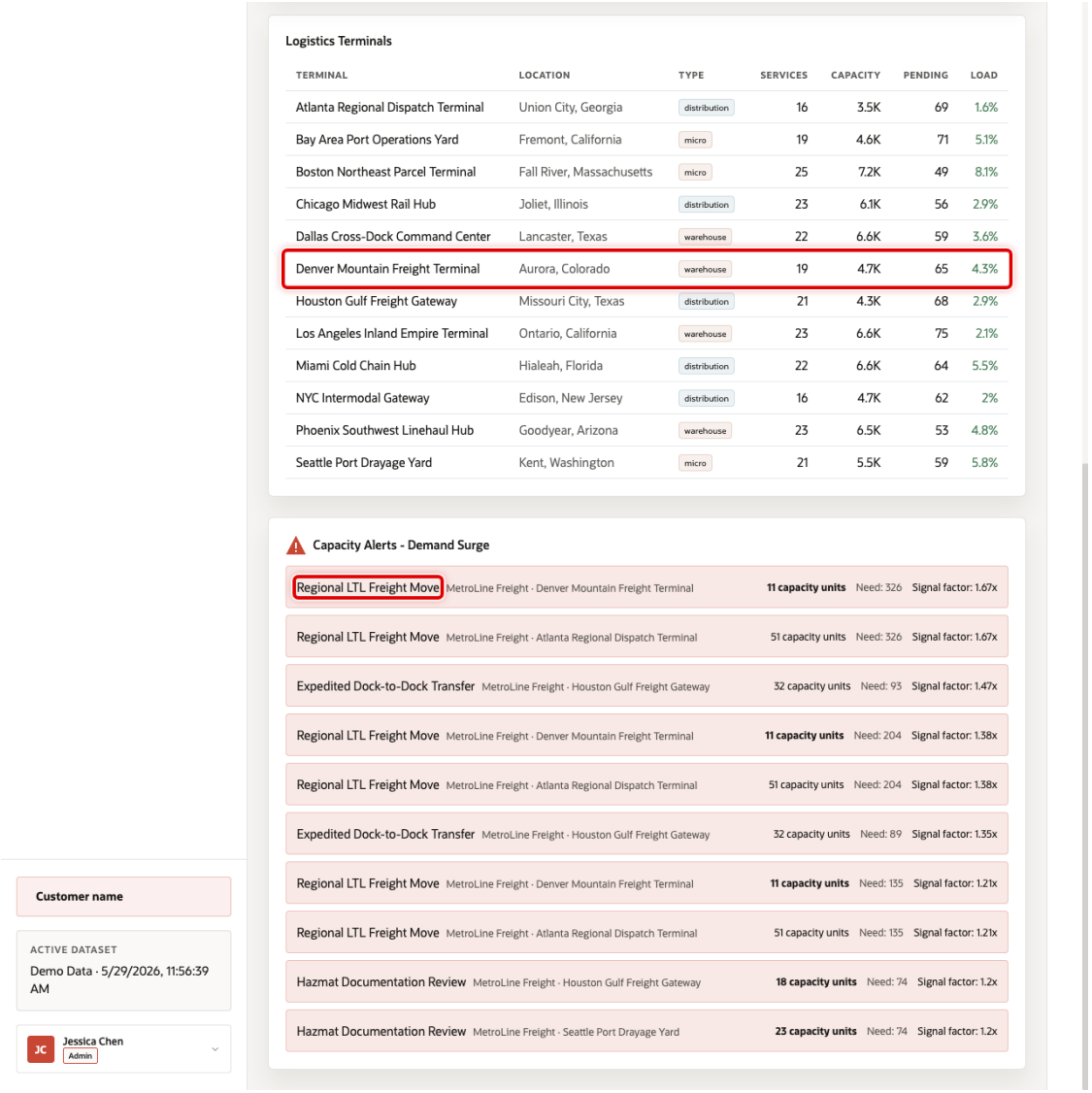


Logistics terminal table with terminal load highlighted

Use the terminal table to connect the map to operational capacity. In the current demo dataset, **Atlanta Regional Dispatch Terminal** shows **3,503** units, **69** pending shipments, and **1.6%** current load, while **Bay Area Port Operations Yard** shows **4,611** units and **71** pending shipments.

1. Scroll to **Logistics Terminals**.
2. Review terminal name, location, type, available capacity, pending shipments, and load percentage.
3. Compare at least two terminals before making a routing recommendation.

Task 4: Investigate a capacity alert



Capacity alert data point for Regional LTL Freight Move at Denver Mountain Freight Terminal

Capacity alerts show where predicted demand may exceed available capacity. In the current demo dataset, **Regional LTL Freight Move** at **Denver Mountain Freight Terminal** is critical, with **11** units on hand, predicted demand above **300** in one forecast row, and freight value at risk.

1. Scroll to **Capacity Alerts**.
2. Review the affected service, terminal, capacity, predicted demand, signal factor, and value at risk.
3. Explain how a dispatcher could use the alert to reroute work, reserve capacity, or escalate the service commitment.

You can move to the next scene.

Credits & Build Notes

- **Author** - Oracle LiveLabs Team
- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-06-09

Scene 7 Shipment Orders & Exceptions

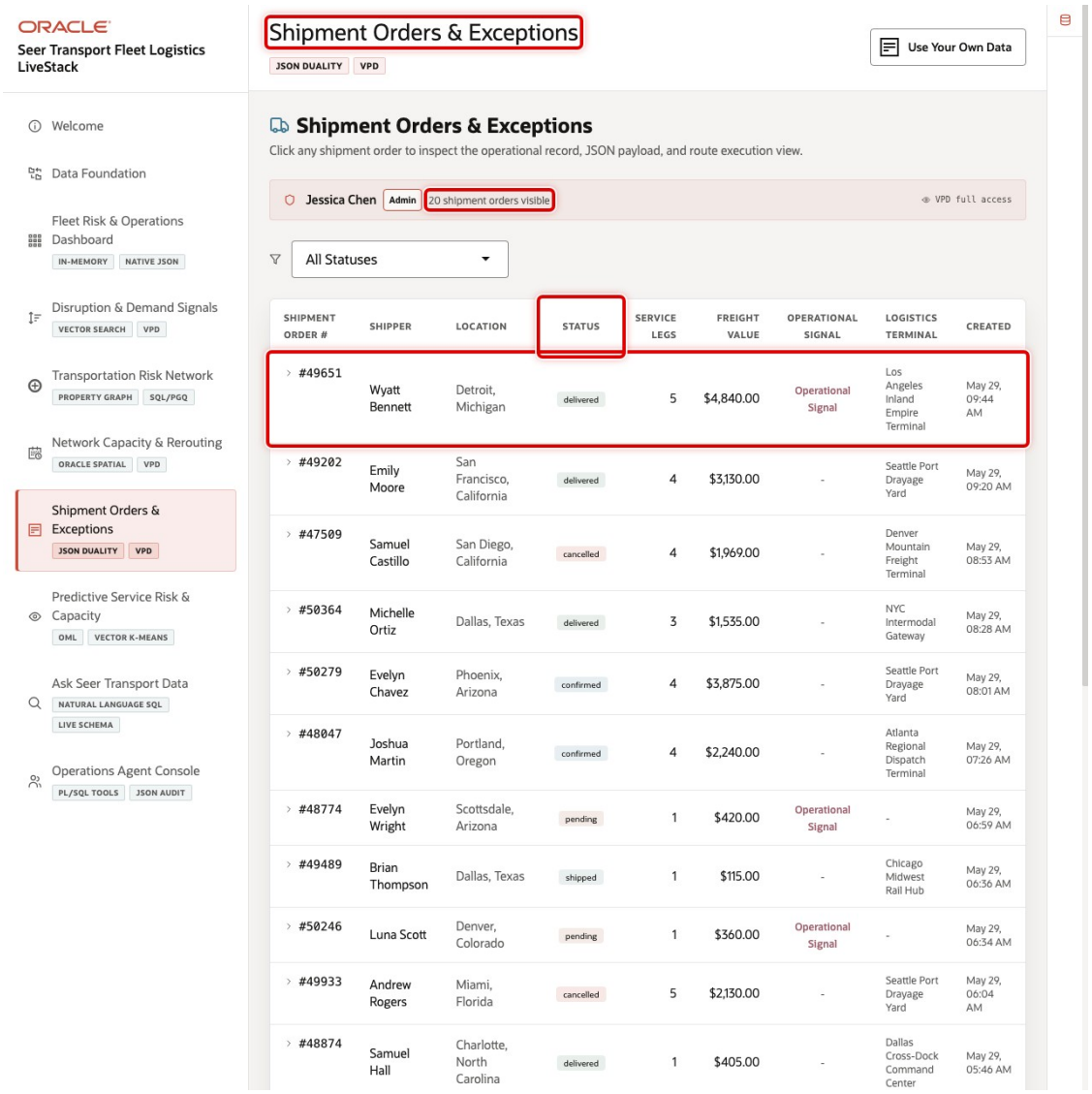
Introduction

Shipment Orders & Exceptions helps operations teams inspect shipment execution, service legs, route context, and JSON duality projections. It connects the operational shipment order list to exception investigation and document-style application access.

Transportation order workflows are often split across dispatch applications, customer systems, carrier tools, and analytics platforms. Business users need relational order detail, while application teams often need document-shaped payloads for APIs and mobile workflows.

Oracle AI Database helps by keeping the same shipment order data available through relational tables and JSON Relational Duality Views. In this scene, the user can inspect a shipment order, review service legs, compare the JSON document shape, see cancellation reasons for cancelled orders, and connect the order back to route and terminal context.

Estimated Time: 10 minutes



Shipment Orders & Exceptions order list with a recent order highlighted

Objectives

In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

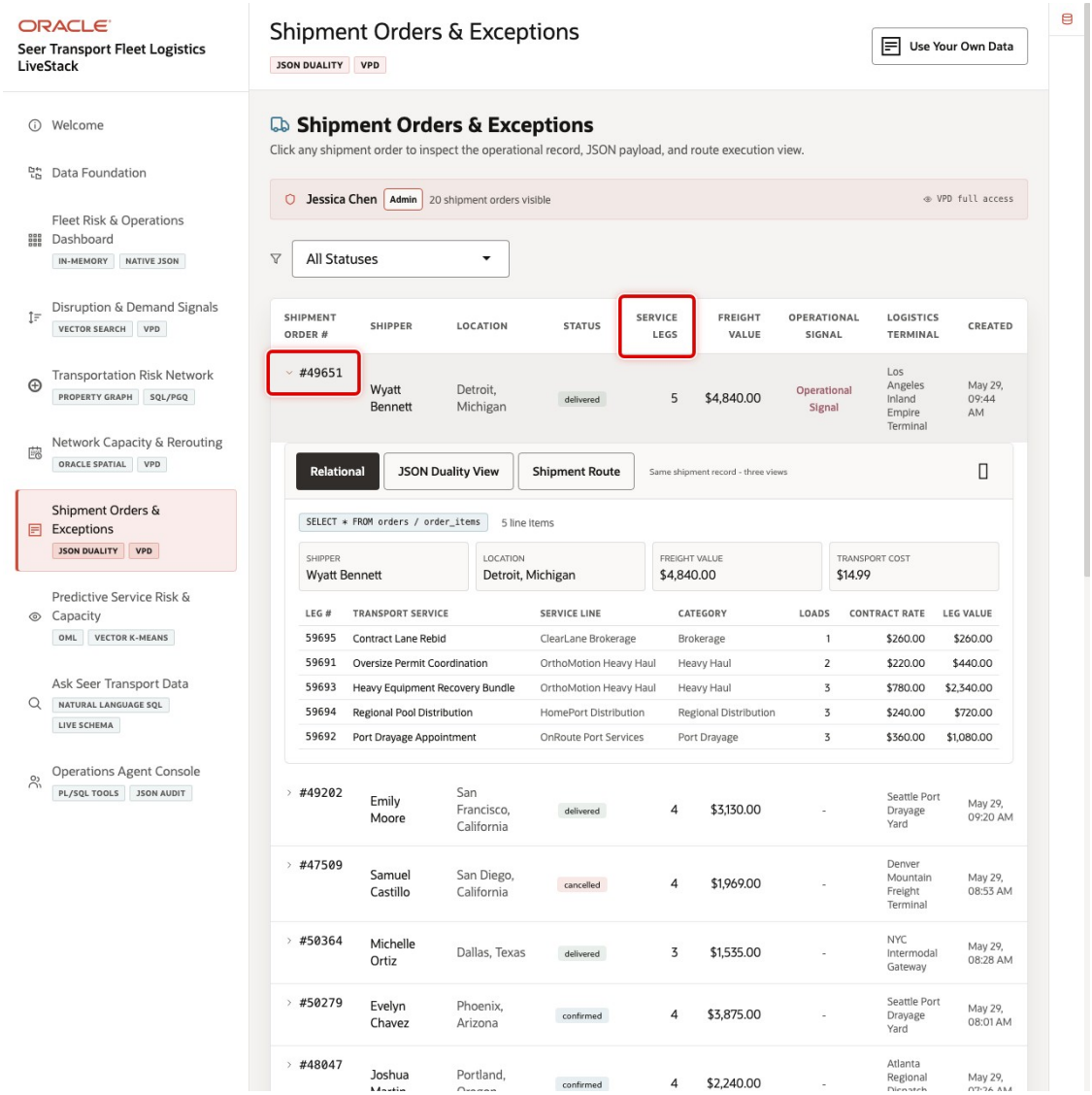
Task 1: Review the shipment order workspace

Use the order list to show shipment execution status and service exposure. In the current demo dataset, order **49651** is a delivered shipment order for **Wyatt Bennett** with **$4,840** in freight value, **5** service legs, and a social-driven source signal.

1. Click **Shipment Orders & Exceptions** in the sidebar.
2. Review the order KPI or summary cards at the top of the page.

3. Review the order list, status filters, and **Reason** column. Cancelled orders now show the operational reason, such as terminal capacity, missed pickup appointment, weather disruption, carrier no-show, or documentation issue.
4. Click order **49651** or another visible high-value shipment order. To discuss recoverable revenue, filter to **Cancelled** and review the reason patterns before opening a row.

Task 2: Inspect the relational shipment detail



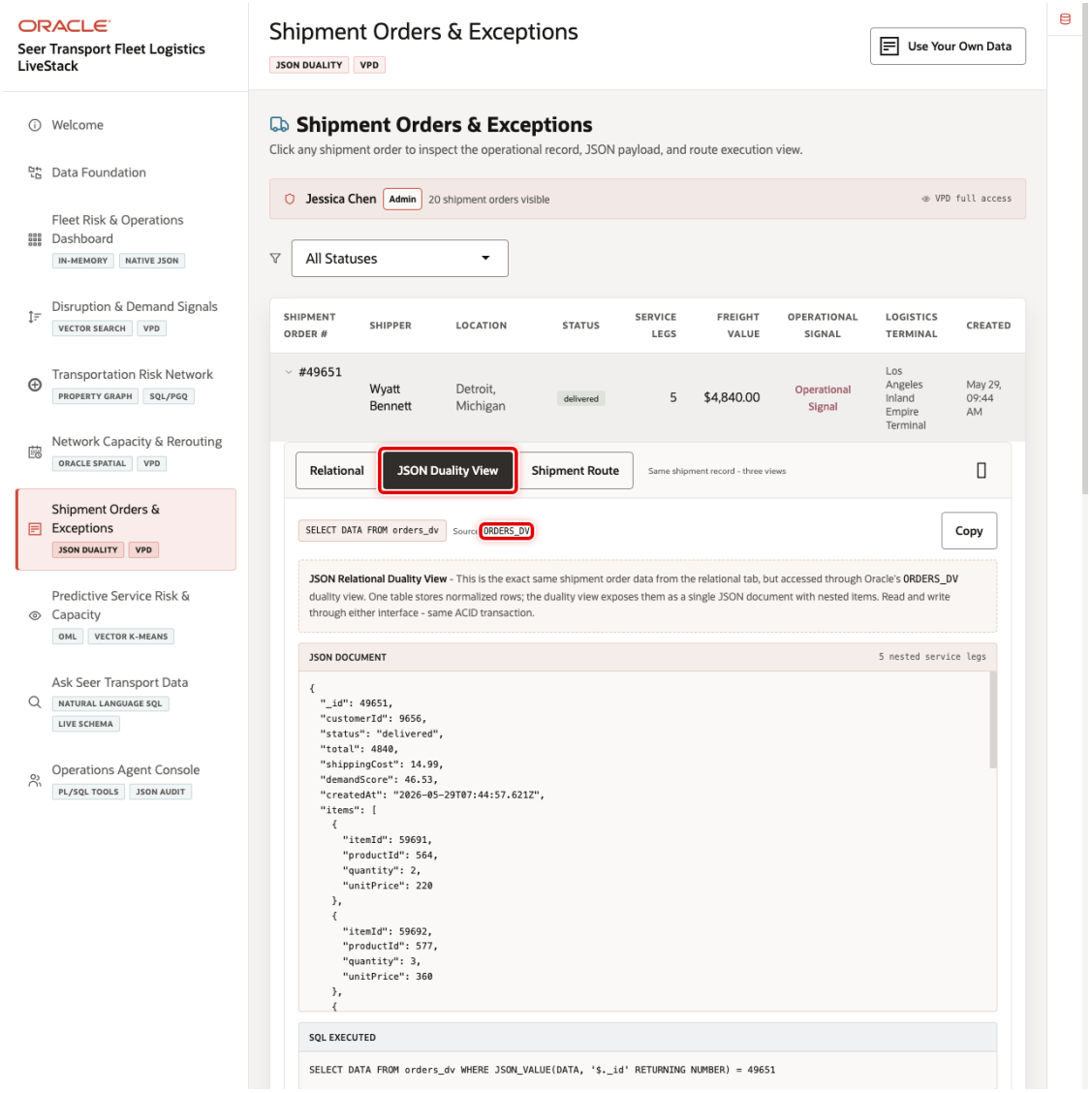
Relational shipment order detail for order 49651

The relational detail shows how the business sees the shipment: shipper, status, service legs, freight value, terminal, carrier, shipment progress, and cancellation reason when the shipment was cancelled.

1. Review the order detail header.

2. Review the service legs. Order **49651** includes **Contract Lane Rebid**, **Oversize Permit Coordination**, **Heavy Equipment Recovery Bundle**, **Regional Pool Distribution**, and **Port Drayage Appointment**.
3. Review shipment progress and carrier details, including **MetroLine Fleet** and tracking number **TRK000000002165**.

Task 3: Compare the JSON Duality View



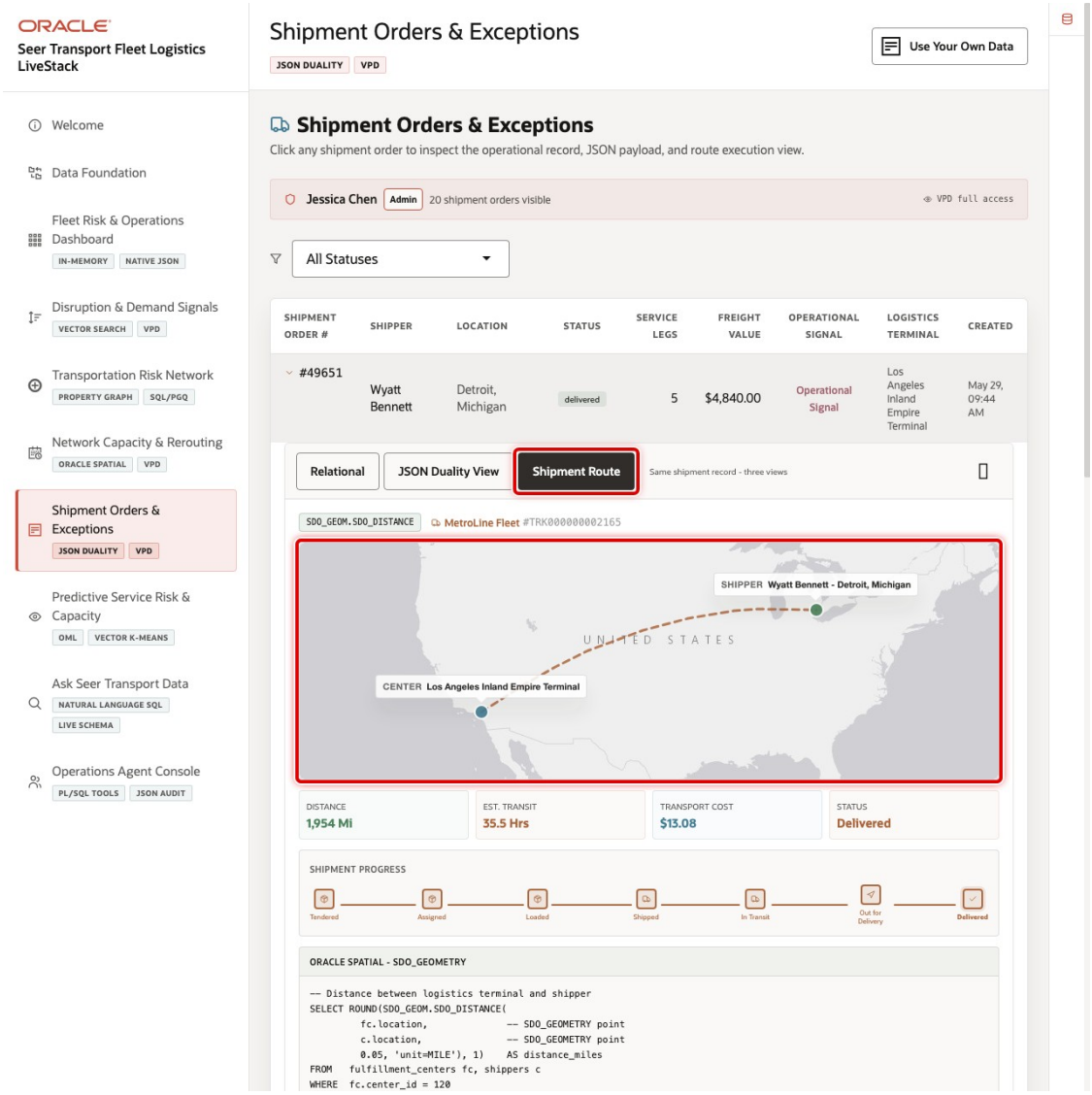
JSON Duality View for shipment order 49651

Use the JSON Duality View to show that the same order data can support both business screens and application payloads.

1. Click **JSON Duality View** in the order detail.
2. Review the nested document shape for order header, service legs, shipper context, shipment details, and cancellation reason when present.

3. Explain that the JSON view and relational view are not separate copies of the order.

Task 4: Review route and terminal context



Shipment route and terminal context for order 49651

Use the route tab to connect the order back to terminal and geography.

1. Click **Shipment Route**.
2. Review the origin terminal and destination context.
3. Compare the shipment route with the capacity and routing scene to show how order execution connects to spatial planning.

You can move to the next scene.

Credits & Build Notes

- **Author** - Oracle LiveLabs Team

- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-06-09

Scene 8 Predictive Service Risk & Capacity

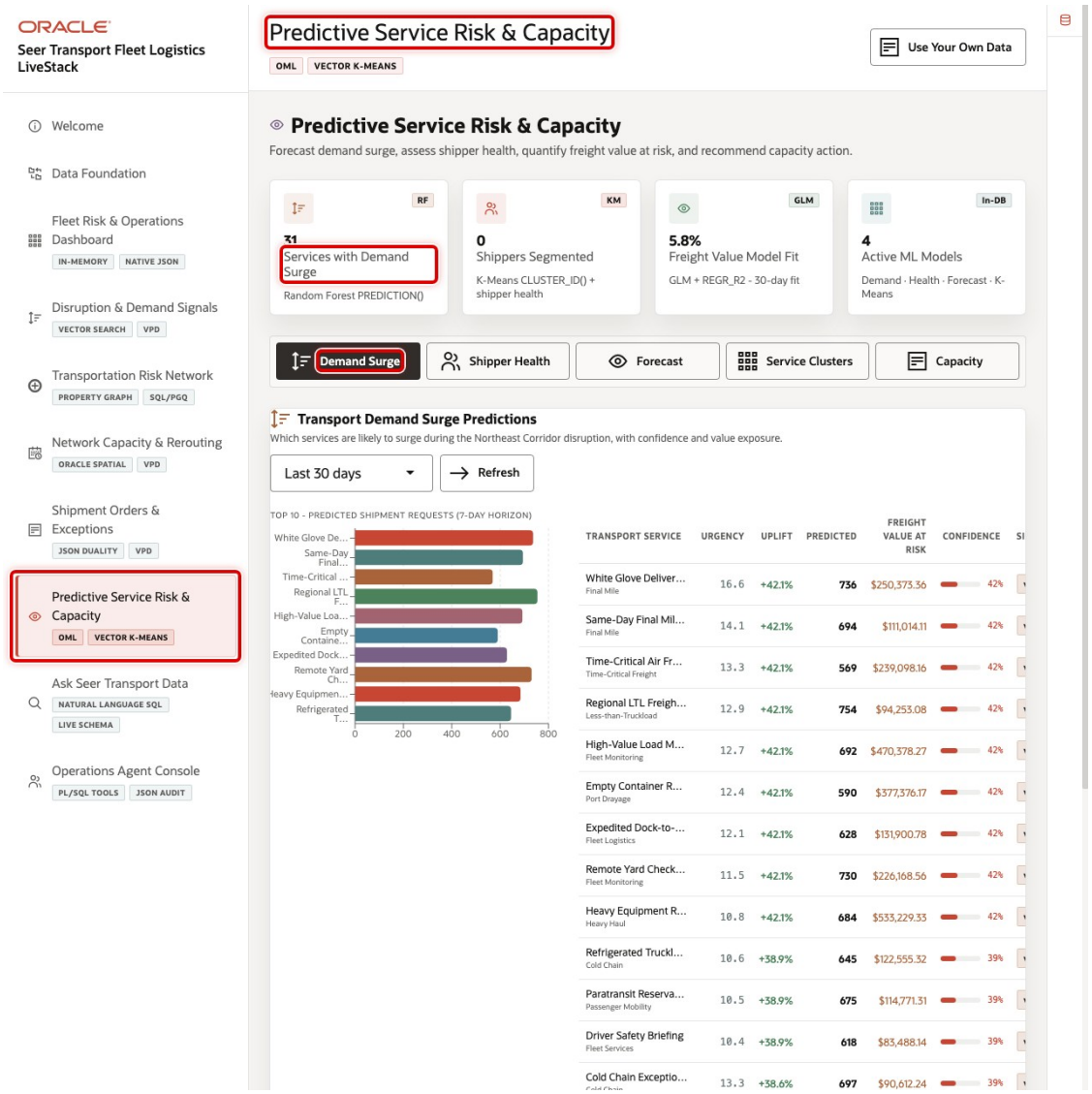
Introduction

Predictive Service Risk & Capacity demonstrates Oracle Machine Learning and analytics workflows for transportation demand, shipper health, freight value forecasting, service similarity clustering, and capacity intelligence.

Transportation teams need to act before service risk becomes visible in missed pickups, terminal backlogs, or customer escalations. That is difficult when predictive models, operational orders, signal feeds, and capacity data are separated across tools.

Oracle AI Database helps by keeping model scoring, SQL analytics, similarity features, forecasts, and operational data together. In this scene, the user can move between predictive tabs and show how model output becomes a planning decision.

Estimated Time: 10 minutes



Predictive Service Risk & Capacity summary cards and mode tabs

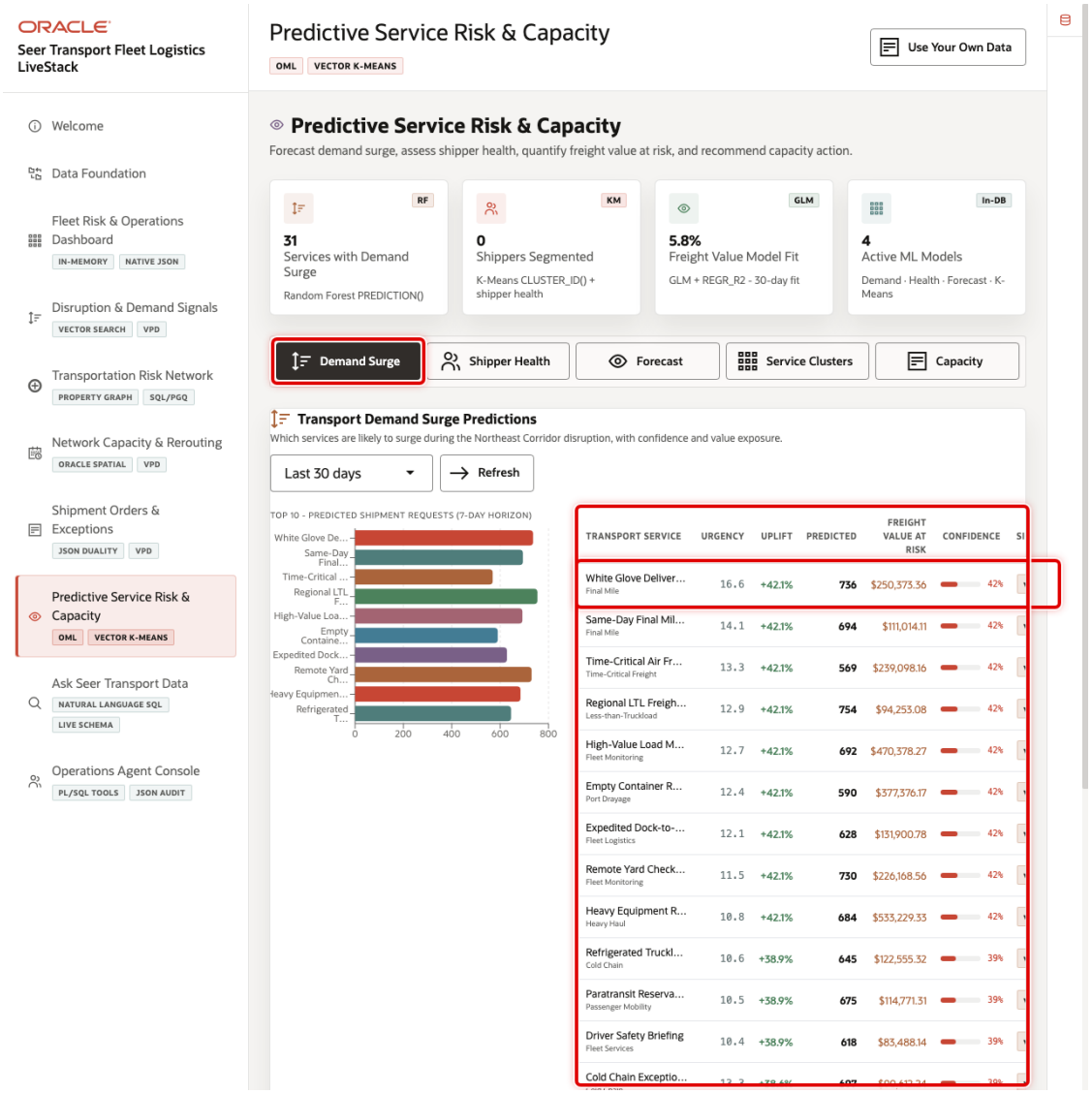
Objectives

In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

Task 1: Review the predictive analytics workspace

1. Click **Predictive Service Risk & Capacity** in the sidebar.
2. Review the summary cards across the top of the page.
3. Review the analytics tabs: **Demand Surge**, **Shipper Health**, **Forecast**, **Service Clusters**, and **Capacity**.
4. Use Oracle Internals after the business flow is clear to explain DBMS_DATA_MINING and SQL scoring.

Task 2: Inspect Demand Surge

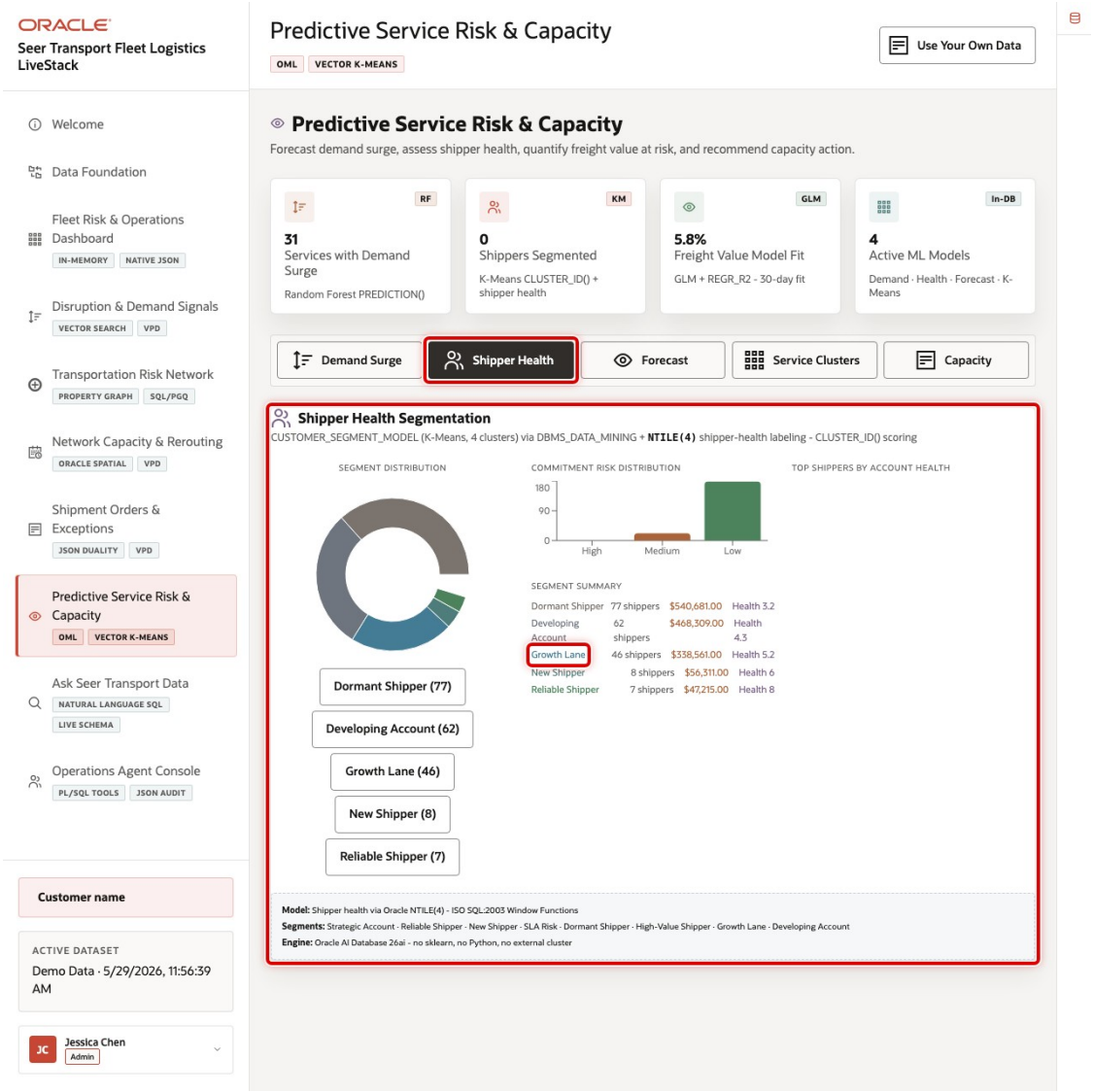


Demand Surge table with top service row highlighted

Demand Surge shows which services are likely to experience elevated demand. In the current demo dataset, **White Glove Delivery Crew** is a top predicted surge service with predicted demand above **700** requests and more than **$250K** in freight value opportunity.

1. Click **Demand Surge**.
2. Review the predicted shipment request chart.
3. Review **Top request reasons** below the chart so the drivers are visible without relying on hover actions.
4. Review the top service rows, confidence, signal label, and freight value at risk.
5. Change the scoring window if you want to compare time horizons.

Task 3: Filter Shipper Health

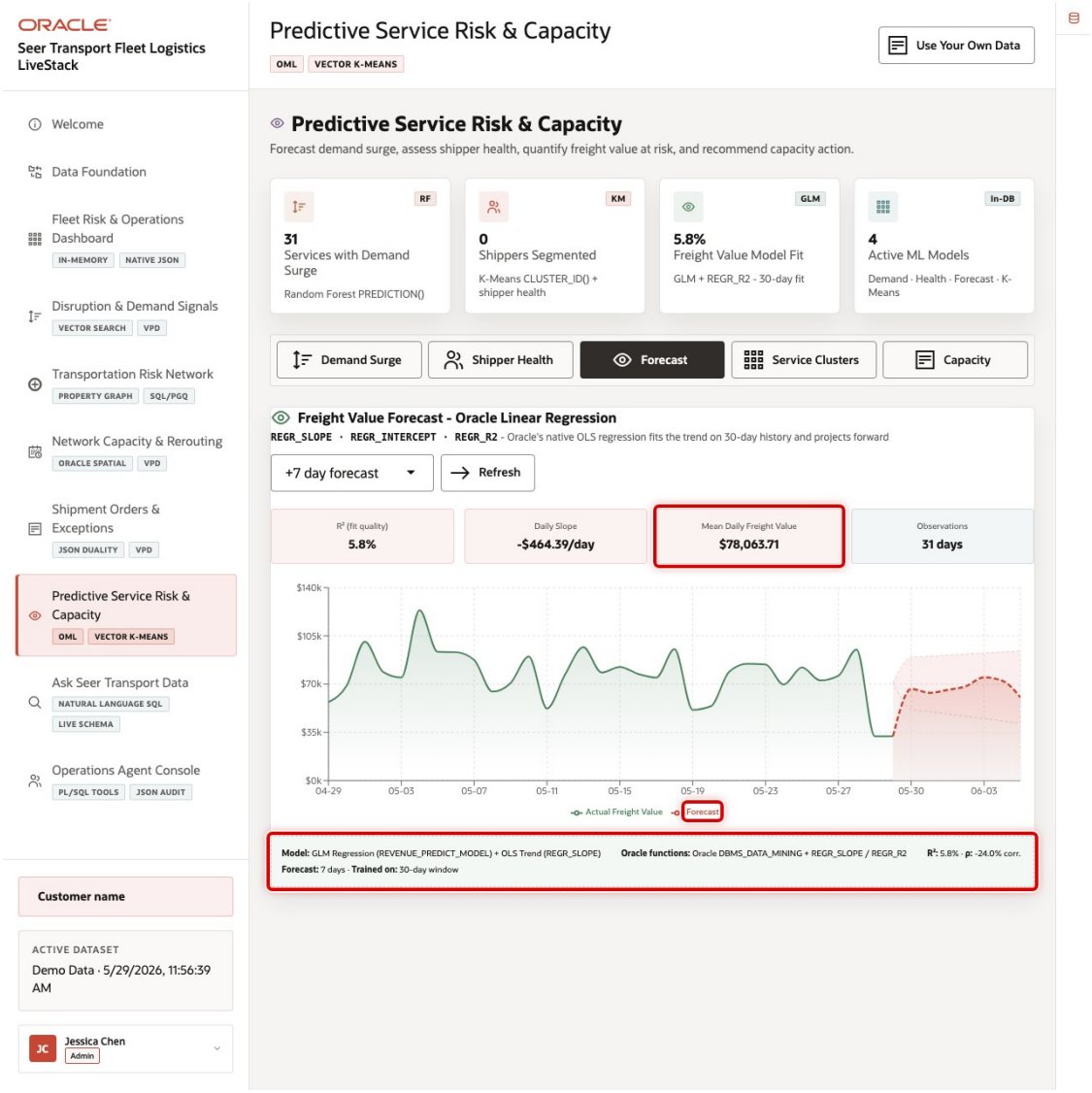


Shipper Health segment filter and shipper list highlighted

Shipper Health helps account and operations teams understand commitment risk and account patterns.

1. Click **Shipper Health**.
2. Review the segment distribution and commitment risk chart.
3. Click a segment filter such as **Growth Lane** or **Developing Account**.
4. Review the filtered shipper list.

Task 4: Change the Forecast horizon

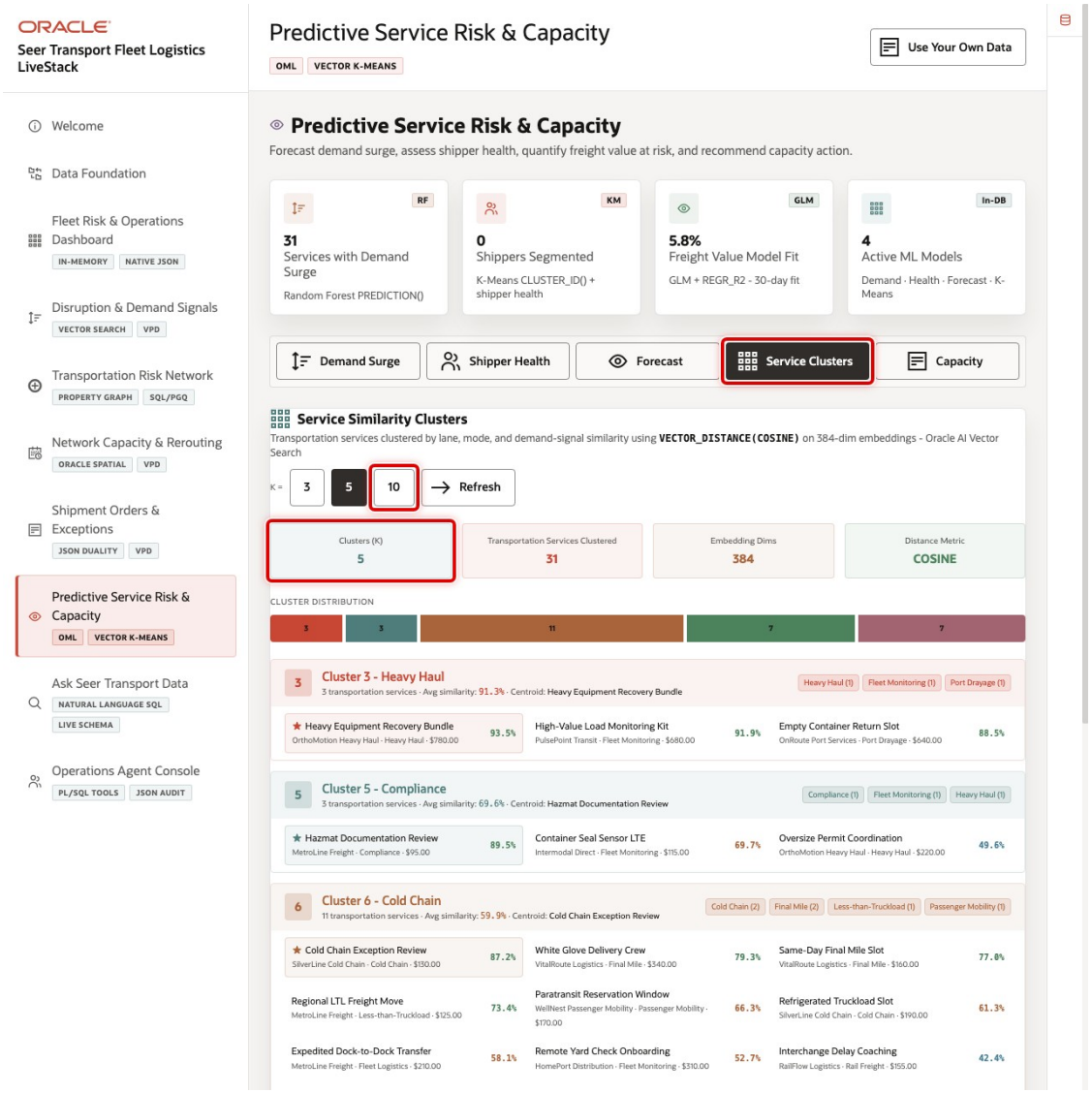


Freight value forecast horizon, model cards, and chart highlighted

The forecast tab shows freight value trend and confidence bands so operators can understand expected service-value movement.

1. Click **Forecast**.
2. Change the forecast horizon using the dropdown.
3. Review model quality, daily slope, mean daily freight value, observations, and the forecast chart.

Task 5: Change Service Clusters

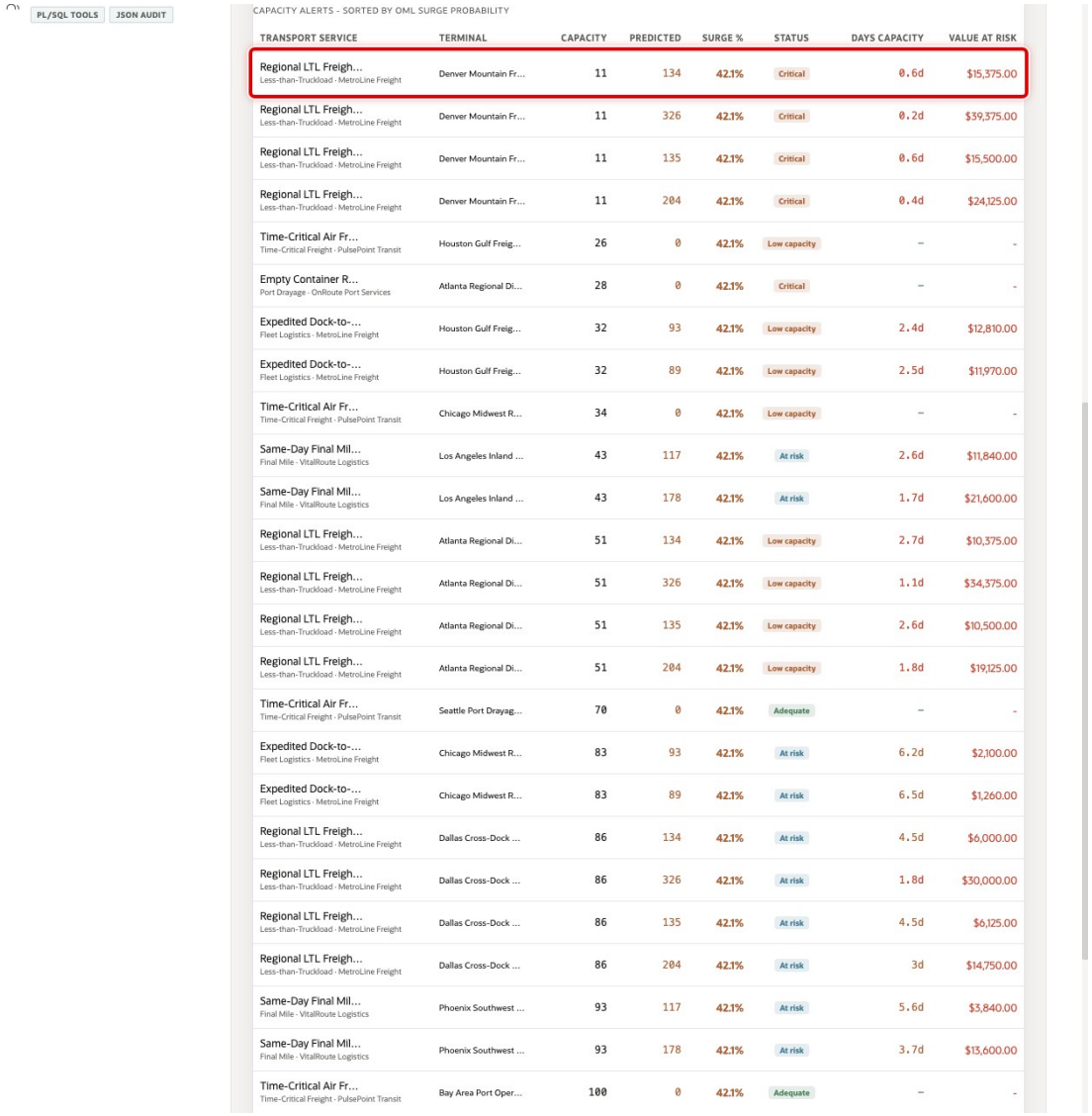


Service cluster controls, metrics, and cluster assignments highlighted

Service clusters group transportation services by similarity so operators can reason about related services and risk cohorts. The visible cards now use business-facing labels such as **Services Compared**, **Similarity Basis**, and **Operational Use**; detailed embedding and distance-metric proof remains in Oracle Internals and SQL evidence.

1. Click **Service Clusters**.
2. Change the K value.
3. Review cluster distribution, top category, centroid service, similarity percentages, and the business-facing cluster summary cards.

Task 6: Review Capacity Intelligence



TRANSPORT SERVICE	TERMINAL	CAPACITY	PREDICTED	SURGE %	STATUS	DAYS CAPACITY	VALUE AT RISK
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Denver Mountain Fr...	11	134	42.1%	Critical	0.6d	$15,375.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Denver Mountain Fr...	11	326	42.1%	Critical	0.2d	$39,375.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Denver Mountain Fr...	11	135	42.1%	Critical	0.6d	$15,500.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Denver Mountain Fr...	11	204	42.1%	Critical	0.4d	$24,125.00
Time-Critical Air Fr... Time-Critical Freight · PulsePoint Transit	Houston Gulf Freig...	26	0	42.1%	Low capacity	–	-
Empty Container R... Port Drayage · OnRoute Port Services	Atlanta Regional Di...	28	0	42.1%	Critical	–	-
Expedited Dock-to-... Fleet Logistics · MetroLine Freight	Houston Gulf Freig...	32	93	42.1%	Low capacity	2.4d	$12,810.00
Expedited Dock-to-... Fleet Logistics · MetroLine Freight	Houston Gulf Freig...	32	89	42.1%	Low capacity	2.5d	$11,970.00
Time-Critical Air Fr... Time-Critical Freight · PulsePoint Transit	Chicago Midwest R...	34	0	42.1%	Low capacity	–	-
Same-Day Final Mil... Final Mile · VitalRoute Logistics	Los Angeles Inland ...	43	117	42.1%	At risk	2.6d	$11,840.00
Same-Day Final Mil... Final Mile · VitalRoute Logistics	Los Angeles Inland ...	43	178	42.1%	At risk	1.7d	$21,600.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Atlanta Regional Di...	51	134	42.1%	Low capacity	2.7d	$10,375.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Atlanta Regional Di...	51	326	42.1%	Low capacity	1.1d	$34,375.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Atlanta Regional Di...	51	135	42.1%	Low capacity	2.6d	$10,500.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Atlanta Regional Di...	51	204	42.1%	Low capacity	1.8d	$19,125.00
Time-Critical Air Fr... Time-Critical Freight · PulsePoint Transit	Seattle Port Drayag...	70	0	42.1%	Adequate	–	-
Expedited Dock-to-... Fleet Logistics · MetroLine Freight	Chicago Midwest R...	83	93	42.1%	At risk	6.2d	$2,100.00
Expedited Dock-to-... Fleet Logistics · MetroLine Freight	Chicago Midwest R...	83	89	42.1%	At risk	6.5d	$1,260.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Dallas Cross-Dock ...	86	134	42.1%	At risk	4.5d	$6,000.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Dallas Cross-Dock ...	86	326	42.1%	At risk	1.8d	$30,000.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Dallas Cross-Dock ...	86	135	42.1%	At risk	4.5d	$6,125.00
Regional LTL Freigh... Less-than-Truckload · MetroLine Freight	Dallas Cross-Dock ...	86	204	42.1%	At risk	3d	$14,750.00
Same-Day Final Mil... Final Mile · VitalRoute Logistics	Phoenix Southwest ...	93	117	42.1%	At risk	5.6d	$3,840.00
Same-Day Final Mil... Final Mile · VitalRoute Logistics	Phoenix Southwest ...	93	178	42.1%	At risk	3.7d	$13,600.00
Time-Critical Air Fr... Time-Critical Freight · PulsePoint Transit	Bay Area Port Oper...	100	0	42.1%	Adequate	–	-

Capacity Intelligence alert table with high-risk rows highlighted

Capacity Intelligence joins demand forecasts with terminal capacity. In the current demo dataset, the page shows **100** monitored service-terminal pairs, critical capacity records, at-risk records, and total freight value at risk.

1. Click **Capacity**.
2. Review critical capacity, SLA risk, OML surge predicted, freight value at risk, and total monitored cards.
3. Review the capacity alert rows sorted by OML surge probability.
4. Focus on rows where predicted demand exceeds available capacity.

You can move to the next scene.

Credits & Build Notes

- **Author** - Oracle LiveLabs Team
- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-06-09

Scene 9 Ask Seer Transport Data

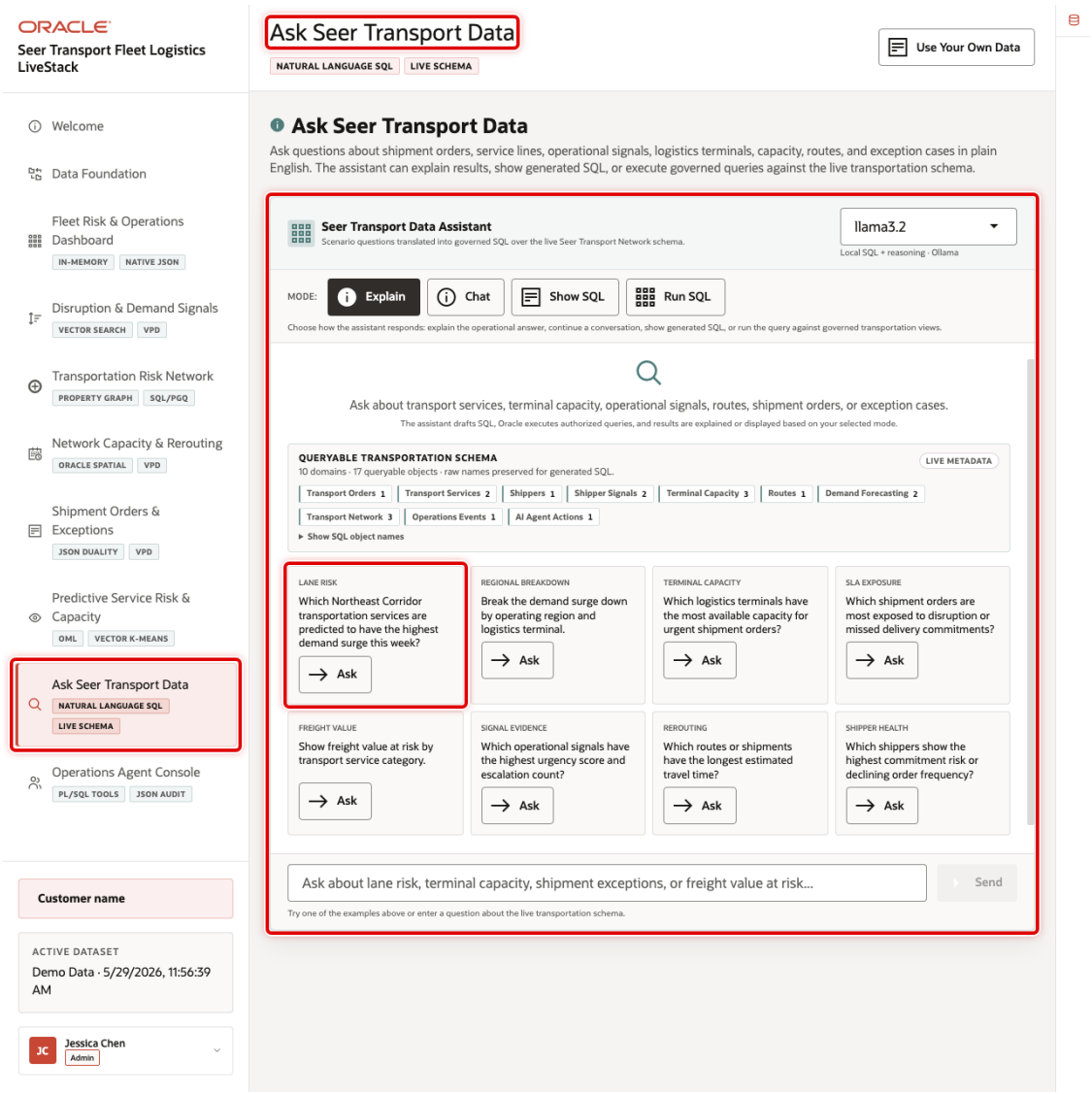
Introduction

Ask Seer Transport Data helps business users ask transportation questions in plain language without losing transparency. Users can inspect generated SQL, run it against trusted Oracle data, and review returned rows, which makes self-service analytics faster and easier to trust.

Natural-language data access can create governance risk if a model generates invalid SQL, references the wrong tables, hides the logic behind an answer, or exposes more data than the user should see. Transportation teams need fast answers, but data teams still need traceability, read-only execution, and a clear source of truth.

Oracle AI Database helps address these challenges by keeping query execution grounded in the live transportation schema. In this LiveStack Demo, the app sends the business question and schema context to the local Ollama runtime, validates the generated SQL path, and uses Oracle AI Database 26ai as the execution authority.

Estimated Time: 10 minutes



Ask Seer Transport Data workspace with modes and example questions highlighted

Objectives

In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

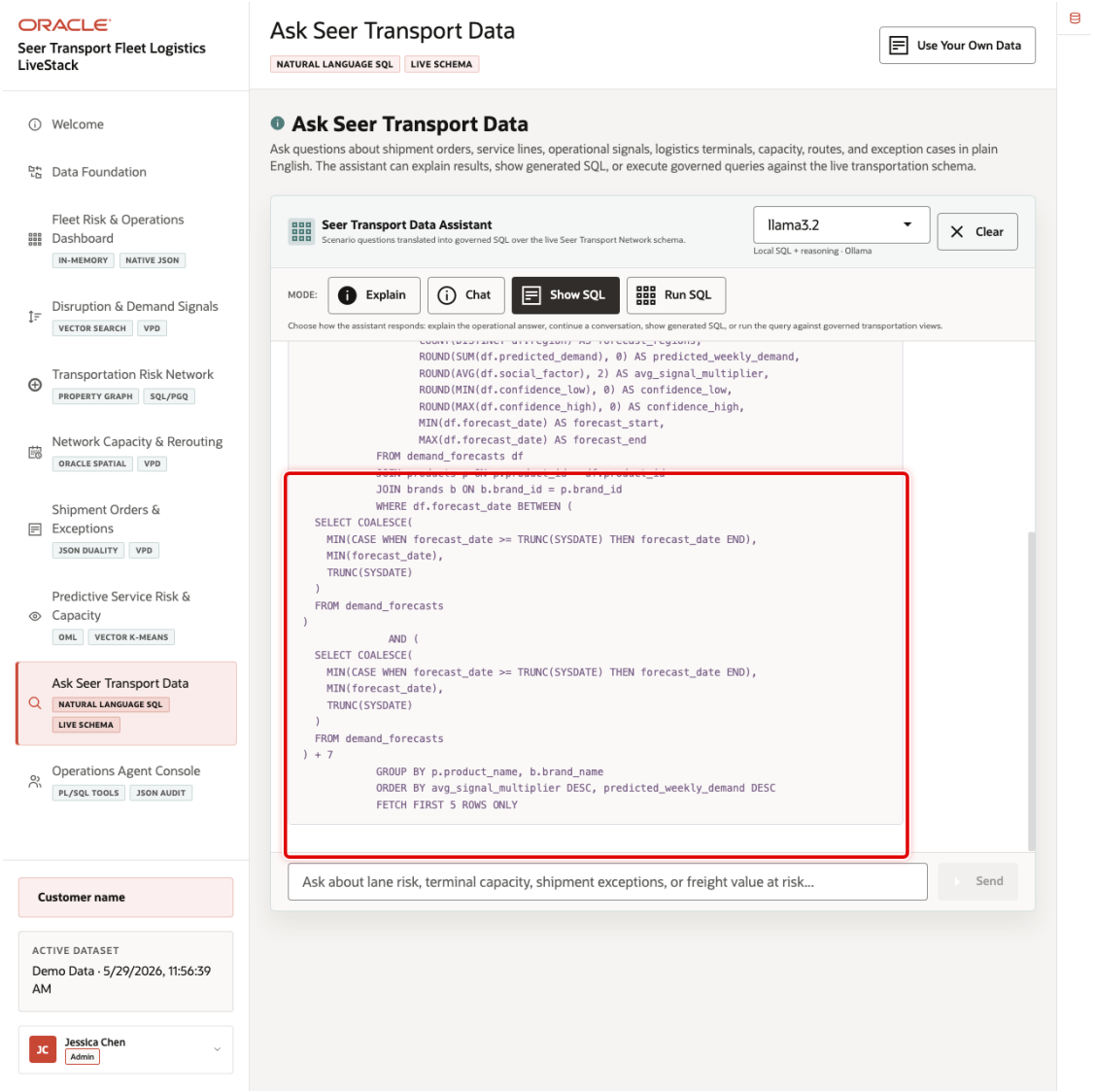
Task 1: Review the Ask Seer Transport Data workspace

Review the workspace to show how business users can ask questions in plain language while still keeping the query path visible and controlled.

1. Click **Ask Seer Transport Data** in the sidebar.
2. Review the runtime profile in the top right of the chat card. The current demo uses the local **llama3.2** runtime through the **SC_LLAMA_PROFILE** profile.
3. Review the four modes: **Explain**, **Chat**, **Show SQL**, and **Run SQL**.

4. Review the example question tiles.
5. Focus on the **Lane Risk** question: **Which Northeast Corridor transportation services are predicted to have the highest demand surge this week?**

Task 2: Inspect generated SQL



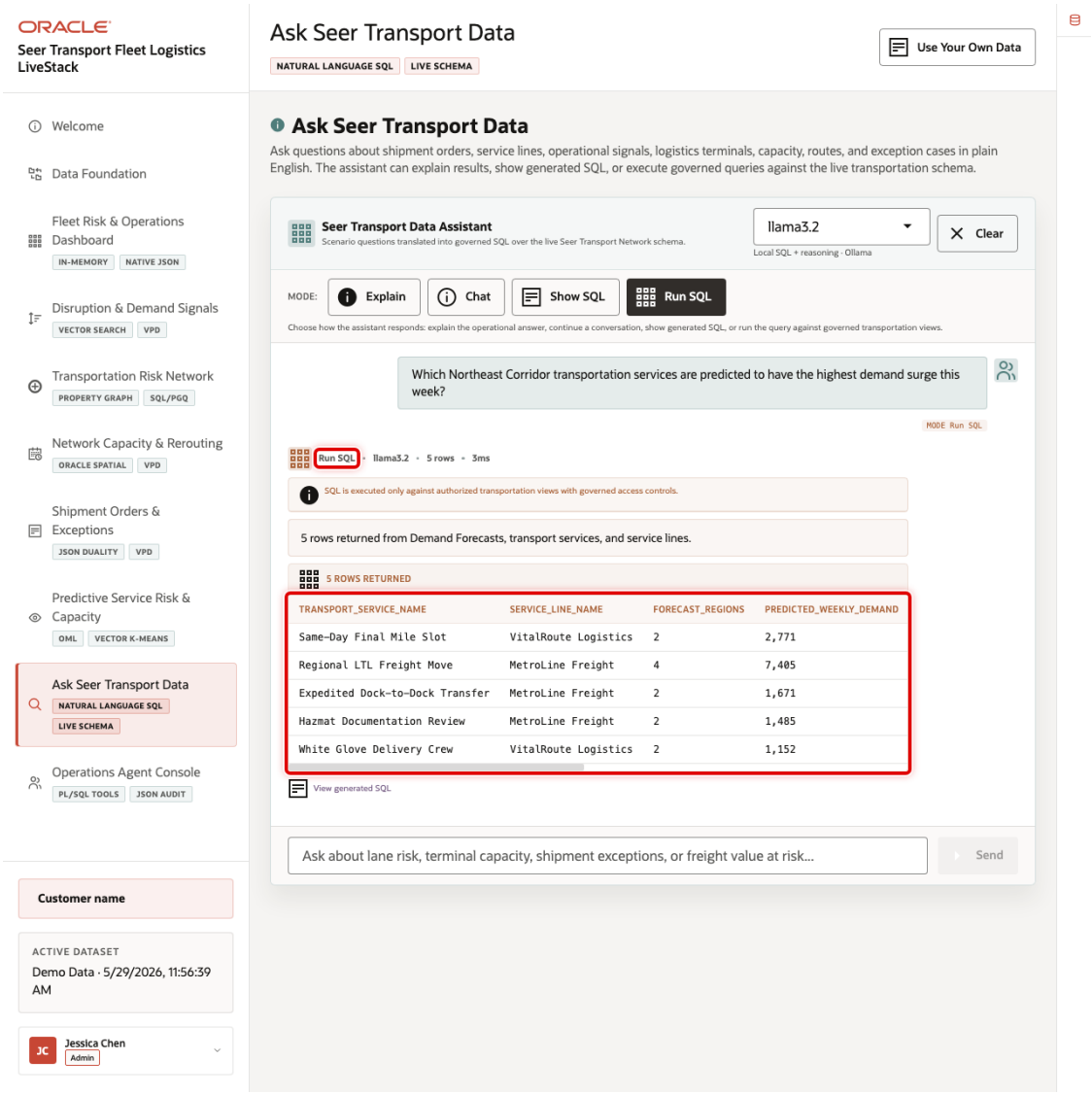
Generated SQL for the Northeast Corridor demand surge question

Inspect the generated SQL to show that the answer is traceable. Even if the user does not read every line, the query can be reviewed instead of trusting a hidden AI response.

1. Click **Show SQL**.
2. Click **Ask** on **Which Northeast Corridor transportation services are predicted to have the highest demand surge this week?**
3. Review the generated SQL.

The generated SQL uses demand forecasts, transportation services, and service lines to find predicted demand surge for the current forecast window. This is the governance moment in the scene: the business user can inspect the query path before asking the database to return rows.

Task 3: Run the SQL and inspect the returned data



Run SQL results for predicted transportation demand surge

Run the SQL and inspect returned rows to find concrete operational pressure.

1. Click **Clear** if the generated SQL result is still visible.
2. Click **Run SQL**.
3. Click **Ask** on the same **Lane Risk** question.
4. Review the returned table.
5. Focus on the first row: **Same-Day Final Mile Slot**.

In the current demo dataset, the question returns **5** rows. The first row is **Same-Day Final Mile Slot** from **VitalRoute Logistics**, with **2,771** predicted weekly demand and an average signal multiplier of **1.82** across forecast regions.

Task 4: Explain the governance pattern

Explain the governance pattern as speed with control: the user asks in plain language, the system shows or runs SQL, Oracle returns trusted data, and the answer remains reviewable.

1. The user asks a transportation question in plain English.
2. The app builds prompt and schema context for the selected runtime profile.
3. Ollama drafts SQL or a response plan.
4. Oracle AI Database executes the generated SQL against the live schema.
5. The UI returns either visible SQL, raw rows, or a narrated answer.

You can move to the next scene.

Credits & Build Notes

- **Author** - Oracle LiveLabs Team
- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-05-29

Scene 10 Operations Agent Console

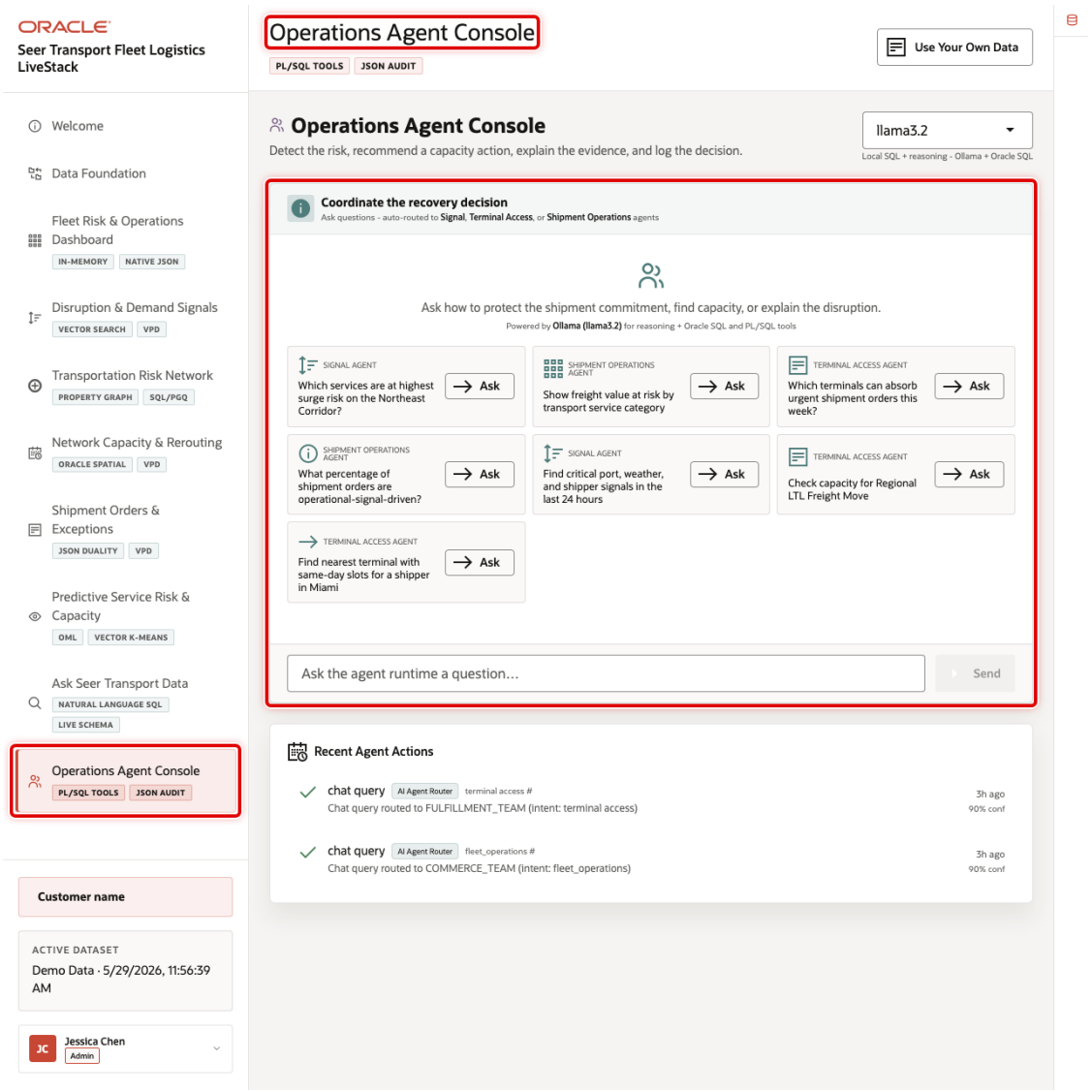
Introduction

Operations Agent Console shows governed AI-assisted operations over the same transportation data foundation. It brings together runtime profile selection, example questions, chat responses, event history, action summaries, and auditable agent actions.

Transportation teams want AI assistance for capacity, route, and service decisions, but recommendations are useful only when they are grounded in trusted data and leave an audit trail. A chat answer that cannot be traced back to a workflow, data source, or action record is difficult to trust in a service-recovery decision.

Oracle AI Database helps by keeping operational data, SQL tools, PL/SQL actions, JSON audit records, and event history close to the same application workflow. In this scene, the user can ask an operational question, review the routed response, and inspect the action audit trail.

Estimated Time: 10 minutes



Operations Agent Console overview with runtime profile, examples, and action summary highlighted

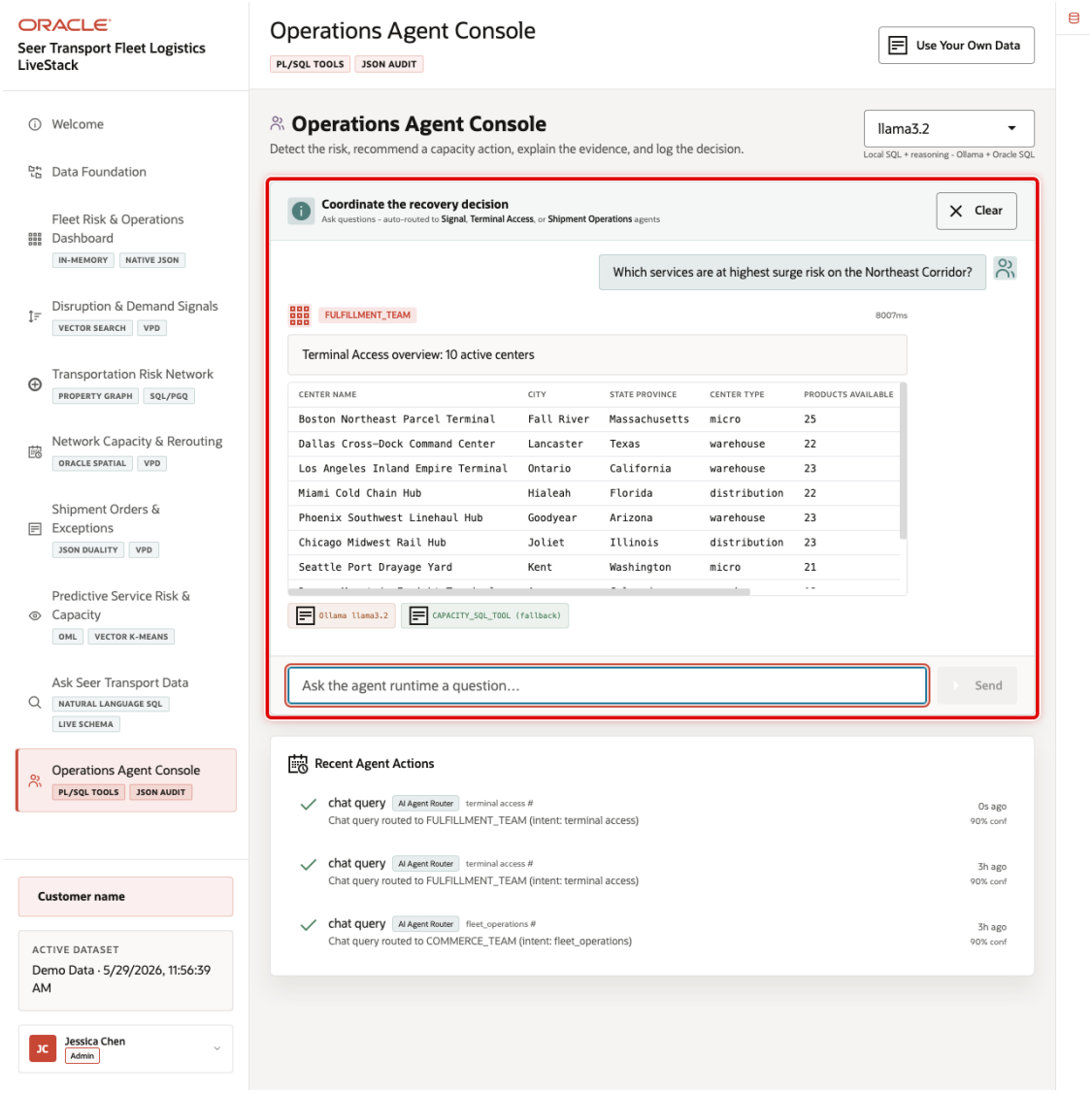
Objectives

In this scene, you will learn what transportation decision the page supports, what evidence the user should inspect, and what action the business may take next.

Task 1: Review the operations agent workspace

1. Click **Operations Agent Console** in the sidebar.
2. Review the active runtime profile selector.
3. Review the example questions for signal, terminal access, and shipment operations teams.
4. Review the action summary and recent action areas.

Task 2: Run a logistics recovery question



Agent response for a Northeast Corridor service-risk question

Run an agent interaction to show how the console turns operational data into a recommendation or explanation.

1. Click the example question **Which services are at highest surge risk on the Northeast Corridor?**
2. Review the agent response, routed team, tool activity, and supporting data.
3. Use **Clear** if you want to reset the chat area before the next question.

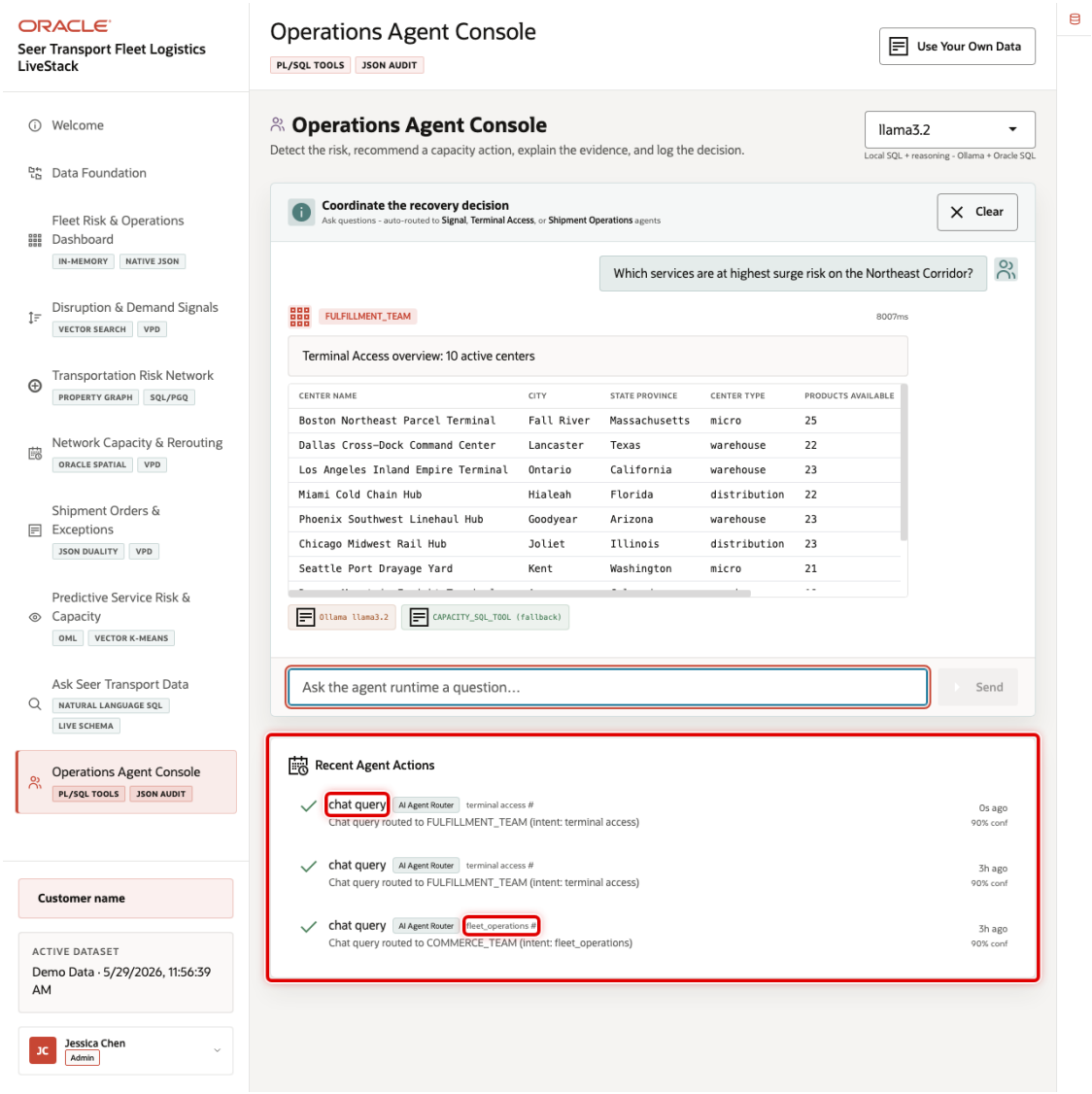
The current demo has existing agent audit records for questions about Northeast Corridor surge risk and freight value at risk. Use those records to explain that the agent workflow is not only conversational; it also records work performed.

Task 3: Interpret the operational story

Use the agent output as the final scene in the logistics journey.

1. Connect the response back to earlier scenes: dashboard KPIs, operational signals, capacity alerts, shipment orders, predictive demand, and Ask Data results.
2. Explain that a human operator can use the answer as a starting point, then open the source scene to inspect the underlying evidence.
3. Emphasize that the agent is part of the operational workflow, not a detached chatbot.

Task 4: Review the agent action audit trail



Recent Agent Actions audit trail with transportation chat query highlighted

1. Scroll to **Recent Agent Actions**.
2. Review the action type, routed team, confidence, status, and timestamp.
3. Focus on the **chat_query** rows for surge-risk or freight-value questions.

4. Explain that Oracle stores the action payload and execution status so the recommendation path remains reviewable.

Credits & Build Notes

- **Author** - Oracle LiveLabs Team
- **Last Updated By/Date** - Oracle LiveLabs Team, 2026-05-29

Take It Home: Run LiveStack with Podman Compose

Introduction

This lab shows how to run a portable LiveStack package in your own environment using Podman Compose.

This guide is intended for technical users who can install Podman and run terminal commands. If you are new to containers, complete the Podman readiness checks before starting. If any readiness check fails, ask your workshop facilitator or system administrator for help before continuing.

Estimated time: 30 minutes for a local run, or 45 minutes when deploying to an OCI Compute VM.

Objectives

In this lab, you will:

- Download the portable LiveStack package.
- Confirm that Podman and Podman Compose are ready.
- Extract the package into a clean local working directory.
- Create a local runtime environment file from `.env.example`.
- Configure proxy settings only when your network requires them.
- Start Oracle Database, ORDS, Ollama, and the application with Podman Compose.
- Validate the application health endpoint and open the local LiveStack.
- Troubleshoot common startup, port, proxy, and first-run issues.
- Optionally deploy the same LiveStack package on an OCI Compute VM.
- Stop the stack cleanly when the demo is complete.

Before you begin

Before starting, confirm that you have:

- Podman installed.
- Podman Compose support available.
    - On macOS or Windows, Podman Desktop is recommended.
    - On Linux, confirm that either `podman compose version` or `podman-compose version` works.

- A running Podman machine on macOS or Windows.
- Terminal access:
  - Terminal on macOS or Linux.
  - PowerShell on Windows.
- Internet access to pull container images and download required application or model assets.
- Enough local resources for database and model containers.
  - Recommended minimum: 8 CPU available for containers.
  - Recommended minimum: 16 GB RAM available for containers.
  - Recommended minimum: 150 GB free disk space.
  - Use larger limits if your package includes larger Ollama models or additional data.
- The required local application port available:
  - `8505` for the LiveStack application.

New to Podman?

Podman is the local container runtime used by this LiveStack package. Podman Compose reads the `compose.yml` file and starts the required services together, including Oracle Database, ORDS, Ollama, and the LiveStack application.

If you are new to Podman, complete these checks before starting the lab.

1. Confirm Podman is installed

```
<copy>
podman --version
</copy>
```

Expected result:

- The command prints a Podman version.

2. Confirm compose support is available

```
<copy>
podman compose version
</copy>
```

If that command does not work, try:

```
<copy>
podman-compose version
</copy>
```

Expected result:

- One of the commands prints a compose version.

3. Start the Podman machine on macOS or Windows

Podman on macOS and Windows runs containers inside a lightweight Linux virtual machine. Start it before running the stack.

```
<copy>
podman machine start
</copy>
```

If the machine has not been created yet, create it first:

```
<copy>
podman machine init
podman machine start
</copy>
```

Expected result:

- The Podman machine starts successfully.

4. Run a quick container test

```
<copy>
podman run --rm hello-world
</copy>
```

Expected result:

- Podman pulls a small test image and prints a success message.

If this test fails, fix Podman setup before continuing with LiveStack.

Task 1: Download the portable package

1. Download the LiveStack package using this link.
2. Save or rename the file as `livestack.zip`.

Expected result:

- You have `livestack.zip` available on your machine.

Task 2: Prepare the working directory

Do not extract or run the stack from your `Downloads` folder. Create a new empty working directory first. The package extracts its files into the current folder, so a clean directory keeps the LiveStack contents organized and avoids Podman issues caused by working from `Downloads`.

For macOS or Linux

1. Open a terminal.
2. Create a new working directory outside of `Downloads`.

```
<copy>
mkdir -p ~/livestack
</copy>
```

3. Move into the new working directory.

```
<copy>
cd ~/livestack
</copy>
```

4. Move the downloaded package from `Downloads` into this directory.

```
<copy>
mv ~/Downloads/livestack.zip .
</copy>
```

5. Extract the package.

```
<copy>
unzip livestack.zip
</copy>
```

6. Confirm that you are in the directory that contains the compose file.

```
<copy>
ls
</copy>
```

   Expected result:

   - You see `compose.yml` or `compose.yaml` in the current directory.

7. Create or refresh the runtime environment file.

```
<copy>
cp .env.example .env
</copy>
```

For Windows

1. Open PowerShell.
2. Create a new working directory outside of `Downloads`.

```
<copy>
New-Item -ItemType Directory -Force -Path "$HOME\livestack" |
Out-Null
</copy>
```

3. Move into the new working directory.

```
<copy>
Set-Location "$HOME\livestack"
</copy>
```

4. Move the downloaded package from `Downloads` into this directory.

   ```
   <copy>
   Move-Item "$HOME\Downloads\livestack.zip" .
   </copy>
   ```

5. Extract the package.

   ```
   <copy>
   tar -xf .\livestack.zip
   </copy>
   ```

6. Move into the extracted LiveStack application folder.

   The folder name can vary by package, such as `finance`, `healthcare`, or another industry name. Replace `<extracted-folder-name>` with the folder name created when you extracted `livestack.zip`.

   ```
   <copy>
   Set-Location .\<extracted-folder-name>
   </copy>
   ```

   Example:

   ```
   <copy>
   Set-Location .\transportation
   </copy>
   ```

7. Confirm that you are in the directory that contains the compose file.

   ```
   <copy>
   Get-ChildItem
   </copy>
   ```

   Expected result:

   - You see `compose.yml` or `compose.yaml` in the current directory.

8. Create or refresh the runtime environment file.

   ```
   <copy>
   Copy-Item .env.example .env -Force
   </copy>
   ```

Expected result:

- You are in a clean working directory outside `Downloads`.
- The extracted package contains `compose.yml` or `compose.yaml`, `.env.example`, application source files, database setup files, and supporting scripts.

- You created a local `.env` file from `.env.example`.

Task 3: Configure proxy settings, if your network requires them

Most home, cloud VM, and unrestricted corporate networks do not need this step. Skip this task when your network allows direct access to container registries, npm, Object Storage, and Ollama model downloads.

1. If your network requires a proxy, configure the proxy in Podman or Podman Desktop before starting the stack. Image pulls happen before the compose containers exist, so Podman itself must be able to reach external registries.

2. Open `.env` and add proxy values if your environment requires them.

   ```
   <copy>
   HTTP_PROXY=http://proxy.example.com:8080
   HTTPS_PROXY=http://proxy.example.com:8080
   http_proxy=http://proxy.example.com:8080
   https_proxy=http://proxy.example.com:8080
   </copy>
   ```

3. Keep local service names in `NO_PROXY` and `no_proxy`.

   ```
   <copy>
   NO_PROXY=localhost,127.0.0.1,::1,db,ords,ollama,app,host.containe
   rs.internal
   no_proxy=localhost,127.0.0.1,::1,db,ords,ollama,app,host.containe
   rs.internal
   </copy>
   ```

Expected result:

- Podman can pull external images through the network proxy when one is required.
- Local compose services still talk to `db`, `ords`, `ollama`, and `app` directly on the internal Podman network.

Task 4: Start the LiveStack with Podman Compose

1. If you are using Podman on macOS or Windows and your Podman machine is not already running, start it now.

   ```
   <copy>
   podman machine start
   </copy>
   ```

2. Confirm that you are in the directory that contains the compose file.

   ```
   <copy>
   ls compose.yml compose.yaml 2>/dev/null
   </copy>
   ```

   On Windows PowerShell, use:

```
<copy>
Get-ChildItem compose.yml,compose.yaml -ErrorAction
SilentlyContinue
</copy>
```

3. Start all services.

   ```
   <copy>
   podman compose up -d --build
   </copy>
   ```

   If your environment uses the standalone `podman-compose` command, use:

   ```
   <copy>
   podman-compose up -d --build
   </copy>
   ```

4. Watch the service state.

   ```
   <copy>
   podman compose ps
   </copy>
   ```

   If your environment uses the standalone `podman-compose` command, use:

   ```
   <copy>
   podman-compose ps
   </copy>
   ```

Expected result:

- `db`, `ords`, `ollama`, and `app` move toward a healthy state.
- The initialization containers complete successfully.
- The application listens on the host port defined by the LiveStack package.

Task 5: Understand first-run downloads

On a clean install, the first startup can take several minutes. The stack pulls container images, installs application dependencies, downloads and warms the configured Ollama models, and loads the ONNX embedding model used by Oracle vector search when the package includes one.

To avoid downloading the ONNX model at first run, place `all_MiniLM_L12_v2.onnx` in `db/data/onnx/` before starting the stack. The database bootstrap uses the packaged file before falling back to the configured `ONNX_MODEL_URL`.

Expected result:

- First-run startup time is expected to be longer than later restarts.
- Subsequent starts reuse the Podman volumes for Oracle data and Ollama models unless you remove the volumes.

Task 6: Validate health and open the application

1. Check the application health endpoint.

   ```
   <copy>
   curl http://localhost:8505/api/health
   </copy>
   ```

2. Open the LiveStack UI in a browser.

   ```
   <copy>
   open http://localhost:8505
   </copy>
   ```

3. If you are not on macOS, open this URL manually in your browser:

   ```
   <copy>
   http://localhost:8505
   </copy>
   ```

Expected result:

- The health check returns a healthy JSON response after Oracle is ready.
- The browser opens the LiveStack locally.

Task 7: Stop the stack when finished

1. Stop and remove running containers while preserving volumes.

   ```
   <copy>
   podman compose down
   </copy>
   ```

   If your environment uses the standalone `podman-compose` command, use:

   ```
   <copy>
   podman-compose down
   </copy>
   ```

2. Use this command only when you intentionally want to delete the local Oracle and Ollama volumes for a complete reset.

   ```
   <copy>
   podman compose down -v
   </copy>
   ```

   If your environment uses the standalone `podman-compose` command, use:

   ```
   <copy>
   podman-compose down -v
   </copy>
   ```

Expected result:

- `podman compose down` stops the local LiveStack cleanly.
- Demo data and downloaded models remain available on the next startup unless you use the explicit volume-removal command.

Troubleshooting

Use this section when the stack does not start, a service is unhealthy, the application does not open, or first startup appears to be slow.

Check service status

```
<copy>
podman compose ps
</copy>
```

If your environment uses the standalone `podman-compose` command, use:

```
<copy>
podman-compose ps
</copy>
```

View service logs

Database logs:

```
<copy>
podman compose logs db
</copy>
```

ORDS logs:

```
<copy>
podman compose logs ords
</copy>
```

Ollama logs:

```
<copy>
podman compose logs ollama
</copy>
```

Application logs:

```
<copy>
podman compose logs app
</copy>
```

Follow application logs in real time:

```
<copy>
podman compose logs -f app
</copy>
```

If your environment uses `podman-compose`, replace `podman compose` with `podman-compose` in the commands above.

Port 8505 is already in use

The LiveStack application uses local port `8505`. If the application does not start or the browser opens another application, check whether another process is already using that port.

For macOS or Linux:

```
<copy>
lsof -i :8505
</copy>
```

For Windows PowerShell:

```
<copy>
netstat -ano | Select-String ":8505"
</copy>
```

Stop the conflicting process or update the compose port mapping if your package supports changing it.

First startup takes a long time

The first run can take several minutes because Podman may need to pull images, install dependencies, download Ollama models, initialize Oracle Database, and load model files. Later starts are usually faster because container images and volumes are reused.

Check progress with:

```
<copy>
podman compose logs -f db
podman compose logs -f ollama
podman compose logs -f app
</copy>
```

Image pulls fail

If image pulls fail, check these items:

- Confirm that your machine has internet access.
- Confirm that Podman can pull a small public test image with `podman run --rm hello-world`.
- If your network requires a proxy, configure the proxy in Podman or Podman Desktop and update `.env` as described in Task 3.
- If you are on a corporate network or VPN, ask your network administrator whether container registry access is allowed.

Health check is not ready

If `curl http://localhost:8505/api/health` does not return a healthy response immediately, wait for the database and application startup to finish, then try again.

Check logs with:

```
<copy>
podman compose logs db
podman compose logs app
</copy>
```

Reset the stack

Stop containers but keep data and downloaded models:

```
<copy>
podman compose down
</copy>
```

Completely reset local data and downloaded models:

```
<copy>
podman compose down -v
</copy>
```

Use the volume-removal command only when you intentionally want a clean reset.

Task 8: Deploy the LiveStack on an OCI Compute VM with Podman (Optional)

Use this task when you want to run the LiveStack on an OCI Compute VM instead of your laptop.

Prerequisites:

- A created Virtual Cloud Network (VCN).
- Security rules configured for the VM subnet.
- Ingress to TCP port `8505` allowed from your approved client CIDR.
- An OCI Compute VM running Oracle Linux 9.
- SSH access to the VM, using the private key paired with the public key added during VM creation.
- The LiveStack package download URL for your workshop.

1. SSH into the OCI VM from your local terminal.

   ```
   <copy>
   ssh -i ~/.ssh/<private-key-file> opc@<vm-public-ip>
   </copy>
   ```

2. Update the VM and install Podman plus common download tools.

```
<copy>
sudo dnf update -y
sudo dnf install -y container-tools podman-docker unzip curl dnf-
plugins-core
podman --version
</copy>
```

3. Install and enable Podman Compose support on Oracle Linux 9.

```
<copy>
sudo dnf install -y oracle-epel-release-el9
sudo dnf config-manager --enable ol9_developer_EPEL
sudo dnf install -y podman-compose
podman compose version || podman-compose version
</copy>
```

4. If the VM firewall is running, allow the LiveStack application port.

```
<copy>
if sudo systemctl is-active --quiet firewalld; then
  sudo firewall-cmd --permanent --add-port=8505/tcp
  sudo firewall-cmd --reload
fi
</copy>
```

5. Create a working directory on the VM.

```
<copy>
mkdir -p ~/livestack
cd ~/livestack
</copy>
```

6. Download the LiveStack package to the VM.

```
<copy>
curl -L -o livestack.zip "<livestack-package-url>"
</copy>
```

7. Extract the package and move into the directory that contains the compose file.

```
<copy>
unzip livestack.zip
COMPOSE_FILE="$(find . -maxdepth 3 \( -name compose.yml -o -name
compose.yaml \) | head -n 1)"
cd "$(dirname "$COMPOSE_FILE")"
</copy>
```

8. Create the runtime environment file.

```
<copy>
cp .env.example .env
</copy>
```

9. Confirm that the compose file exposes the application on port `8505`.

   ```
   <copy>
   grep -n "8505" compose.yml compose.yaml 2>/dev/null
   </copy>
   ```

   The application should be exposed on port `8505`. To reach it from your browser, both the OCI subnet security rules and the VM firewall must allow inbound TCP traffic on port `8505` from your approved client CIDR.

10. Start the LiveStack with Podman Compose.

    ```
    <copy>
    podman compose up -d --build || podman-compose up -d --build
    </copy>
    ```

11. Watch the service state from the VM.

    ```
    <copy>
    podman compose ps || podman-compose ps
    </copy>
    ```

12. Validate the application health endpoint from the VM.

    ```
    <copy>
    curl http://localhost:8505/api/health
    </copy>
    ```

13. Open the public LiveStack URL from your browser.

    ```
    <copy>
    http://<vm-public-ip>:8505
    </copy>
    ```

Expected result:

- The OCI VM is running the LiveStack with Podman Compose.
- The application health endpoint responds locally on the VM.
- The LiveStack UI is reachable from your browser at `http://<vm-public-ip>:8505`.

Why this matters

A portable LiveStack runbook turns this guide into something teams can reproduce instead of just read. By shipping the application as a Podman Compose package with a clear startup flow, you reduce environment drift, make scene validation repeatable, and give teams a practical way to explore the same Oracle-backed experience on macOS, Linux, Windows, and OCI Compute.

This guide is self-service for technical users who can install Podman and run terminal commands. Users who are new to containers should complete the Podman readiness checks before starting and ask for help if those checks fail.

Credits and build notes

- **Author** - LiveLabs Team
- **Last Updated By/Date** - LiveLabs Team, 2026-06-04